

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему

**ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ ИГРУ
«POKER DUEL»**

БГУИР КП 6-05-0612-01 009 ПЗ

Студент

Д. Г. Гетманчук

Руководитель

С. В. Болтак

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Сравнительный анализ существующих решений	7
1.2 Постановка задачи	9
2 Проектирование программы	10
2.1 Проектирование структуры программы	11
2.2 Проектирование системы сетевого взаимодействия	12
3 Программная реализация	13
3.1 Описание разработанных модулей	14
3.2 Описание сетевого модуля	15
4 Тестирование	16
5 Руководство пользователя	18
Заключение	28
Список используемых источников	29
Приложение А (обязательное) Листинг кода с комментариями	30
Приложение Б (обязательное) Схема алгоритма работы системы.....	57

ВВЕДЕНИЕ

В эпоху стремительного развития цифровых технологий традиционные настольные и карточные игры претерпевают масштабную цифровую трансформацию. Игровая индустрия стремительно смещает фокус в сторону веб-приложений, которые предоставляют пользователям мгновенный доступ к игровому процессу без необходимости скачивания и установки тяжеловесного программного обеспечения. Особый интерес представляют соревновательные карточные игры в режиме реального времени, требующие от архитектуры веб-приложения высокой производительности, минимальной задержки при передаче данных и надёжной синхронизации состояний между клиентами.

Разработка качественного игрового веб-приложения сопряжена с необходимостью решения целого комплекса сложных инженерных задач, связанных с организацией сетевого взаимодействия. Использование классического протокола *HTTP* не позволяет обеспечить требуемый уровень интерактивности из-за постоянных накладных расходов на установку соединений. В связи с этим возникает необходимость внедрения современных технологий двустороннего обмена данными в режиме реального времени, эффективных алгоритмов автоматического расчёта игровых комбинаций, а также проектирования отказоустойчивой серверной логики, способной изолированно обрабатывать игровые события.

Для реализации программы поставлены следующие задачи:

- разработать серверную часть на языке программирования *Go* с целью параллельной обработки множества комнат;
- организовать полнодуплексное сетевое соединение между игроками и сервером на основе протокола *WebSocket* [1];
- разработать надёжное реляционное хранилище данных в СУБД *PostgreSQL* для ведения профилей игроков [2];
- создать минималистичный пользовательский интерфейс с применением современных *CSS*-анимации [3].

Актуальность данной курсовой работы обусловлена растущим спросом на кроссплатформенные развлекательные веб-сервисы и необходимостью исследования подходов к созданию высоконагруженных систем на базе современного технологического стека. В качестве объекта разработки выступает сетевая игра «*Poker Duel*», сочетающая в себе классические правила покера в формате дуэли и современные стандарты веб-интерфейсов. . Целью работы является создание клиент-серверного веб-приложения для проведения карточных матчей для двух участников с обеспечением мгновенного обмена данными и валидацией всех действий на стороне сервера.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Предметная область проекта охватывает процессы сетевого взаимодействия, синхронизации данных и контроля правил в карточных онлайн-играх. Деятельность включает в себя управление виртуальными пространствами, распределение игровых ресурсов и фиксацию этапов раздачи карт. Эффективная работа подобных систем базируется на программном обеспечении, способном мгновенно обновлять состояние стола для всех подключённых клиентов. Создание защищённой серверной архитектуры является необходимым условием для верификации действий пользователей и предотвращения модификации данных.

Информационную основу области составляет логический модуль, консолидирующий сведения о состоянии колоды, картах участников и истории сделанных ставок. Система обеспечивает непрерывное хранение параметров текущего раунда, последовательности ходов соперников и распределения банка внутри каждого активного лобби. Интеграция этих данных с сетевым ядром приложения позволяет безошибочно вычислять силу возникающих комбинаций и определять право очерёдности хода. Это формирует надёжный вычислительный базис для оценки игровых ситуаций без использования клиентских методов обработки.

Технологический аспект управления сессией требует наличия интерфейсов, максимально оптимизированных под динамику игрового процесса. Данная область подразумевает обязательную реализацию механизмов для выбора размера ставок, отслеживания лимита времени на какое-либо решение и визуализации компонентов [4]. Спроектированная визуальная среда позволяет участникам полностью сосредоточиться на тактике матча, сокращая временные затраты на ожидание отклика от сервера. Гибкость такой системы обеспечивает быструю реакцию элементов управления на любые действия игроков.

Специфика управления сетевой сессией заключается в жёсткой зависимости от стабильности соединения и точности синхронизации параллельных потоков данных в реальном времени. Программная реализация в рамках подобных проектов успешно решает задачи по автоматическому расчёту выигрышных сочетаний карт и верификации входящих пакетов. Внедрение чётких алгоритмов оптимизирует структуру серверной логики и позволяет проводить точную оценку действий пользователей на основе заложенных правил игры. Итоговое решение объединяет перечисленные аспекты в единую систему, формируя программный комплекс с надёжной архитектурой обмена данных.

1.1 Сравнительный анализ существующих решений

Проектирование игрового веб-приложения требует обязательного мониторинга доступных аналогов. Современный рынок предлагает решения от простых скриптов до тренажёров с математическими расчётами. Изучение популярных платформ позволяет выделить их главные функциональные преимущества, оценить удобство взаимодействия с интерфейсом и сформировать требования к разработке собственного веб-приложения.

Первым рассмотренным аналогом выступает платформа *GTO Wizard*, ориентированная на симуляцию игровых ситуаций на основе теории игр. Сервис позволяет пользователям тестировать различные математические решения в режиме реального времени и детально изучать диапазоны стартовых рук. Однако интерфейс перегружен большим количеством статистических таблиц, что затрудняет его быстрое освоение.



Рисунок 1.1 – Интерфейс «GTO Wizard»

Изучение этой платформы показывает, что подобные системы часто перегружены избыточными данными, снижающими оперативность работы пользователя. Главным недостатком аналогов является отсутствие единой среды, совмещающей лёгкий дизайн и защищённое управление сессиями. Разрабатываемое веб-приложение призвано устранить эти проблемы за счёт использования изолированных программных модулей.

Вторым исследованным решением является специализированный веб-тренажер *PokerHeads*, предназначенный для проведения быстрых тренировочных сессий в формате один на один. Программа обеспечивает базовую визуализацию игрового пространства, фиксирует общую историю совершенных ставок и предоставляет пользователям автоматические подсказки непосредственно во время совершения хода. Главный упор в системе сделан на максимальное упрощение тренировочных циклов и минимизацию времени ожидания между раздачами карт.

Основным недостатком данного аналога является перегруженность клиентской части тяжёлыми графическими скриптами. Это увеличивает время загрузки страниц в браузере и приводит к нестабильной работе интерфейса при слабом сетевом соединении. Кроме того, закрытая архитектура ядра не позволяет адаптировать существующие алгоритмы под кастомные форматы турнирных комнат.

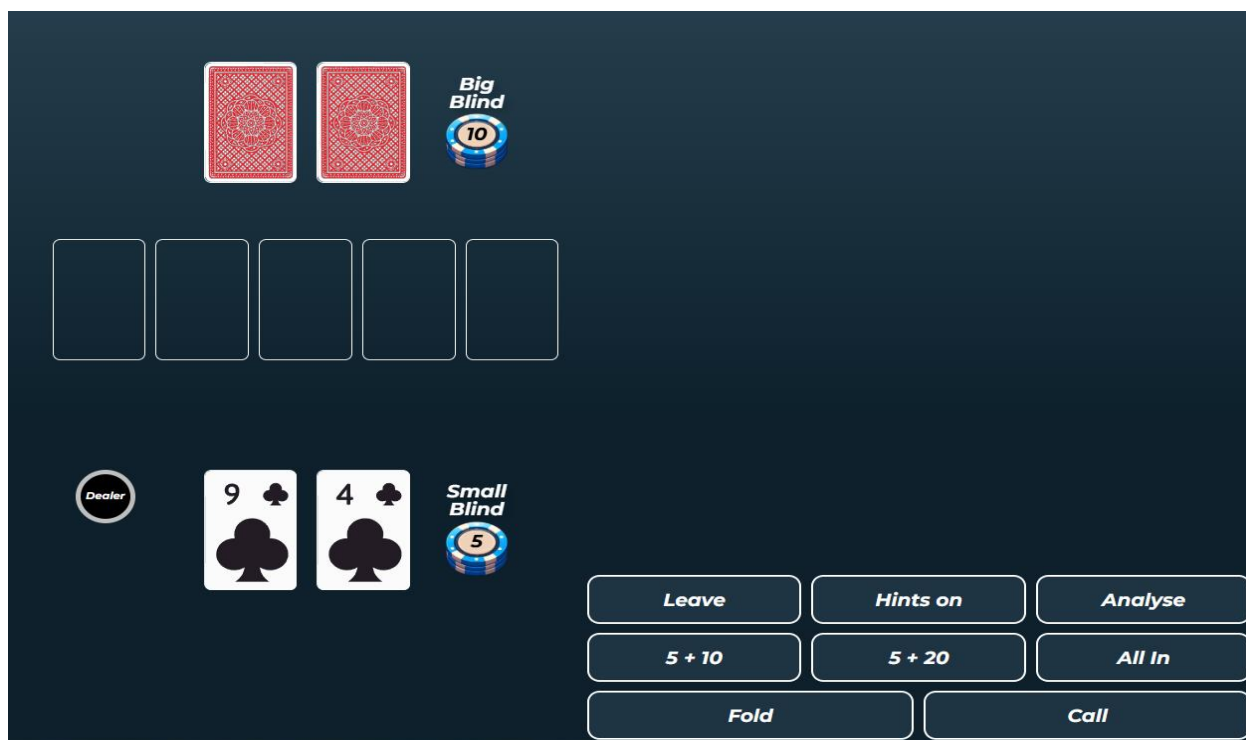


Рисунок 1.2 — Интерфейс «PokerHeads»

Анализ интерфейса *PokerHeads* подтверждает необходимость разработки оптимального дизайна карточных веб-приложений. Наличие избыточных кнопок для вызова подсказок снижает общую динамику проведения матча и отвлекает участников от текущей ситуации. Разрабатываемое веб-приложение нацелено на устранение данных недостатков за счёт внедрения минималистичного оформления.

1.2 Постановка задачи

Данный курсовой проект посвящён разработке программного обеспечения для ведения игрового процесса карточного онлайн-сервиса в режиме реального времени. Основная цель проекта заключается в создании эффективного интерактивного веб-приложения, предназначенного для оптимизации процессов сетевого взаимодействия пользователей во время игровых сессий. В рамках реализации системы предполагается внедрение функционала, позволяющего минимизировать задержки при сетевом обмене данными, обеспечить устойчивое управление турнирными матчами формата один на один, а также гарантировать высокую отказоустойчивость серверной части при нагрузках.

Функции и возможности разрабатываемой программы:

- ведение хранилища данных в *PostgreSQL* для регистрации и структурирования профилей [2];
- организация полнодуплексного сетевого соединения между игроками и сервером на основе *WebSocket* [1];
- автоматический расчёт и валидация игровых комбинаций на стороне сервера с целью исключения ошибок;
- параллельная обработка множества игровых комнат за счёт масштабируемой серверной части на *Go*;
- визуализация игрового процесса с применением *CSS*-анимаций для повышения интерактивности интерфейса [3][4].

Программа предназначена для автоматизации работы игровых комнат, оптимизации времени обработки сетевых запросов и обеспечения стабильного взаимодействия участников в процессе матча. Разрабатываемый проект позволяет эффективно распределять нагрузку на сервер при одновременной активности множества независимых лобби. Применение централизованных алгоритмов на стороне сервера гарантирует корректную обработку входящих пакетов данных, выполнение расчётов в строгом соответствии с правилами игры и синхронизацию текущего игрового состояния между подключёнными клиентами.

Особое внимание в проекте уделяется удобству пользовательского интерфейса, обеспечивающего оперативный доступ к ключевым метрикам игрового стола. Также учитывается гибкость системы по отношению к различным форматам соревновательных комнат, что позволяет адаптировать веб-платформу под индивидуальные цели участников. Реализация проекта способствует систематизации подходов к управлению изолированными сессиями, повышению прозрачности сетевых процессов и формированию надёжной базы для дальнейшего расширения функциональных возможностей разрабатываемого программного обеспечения.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

Проектирование является ключевым этапом разработки, определяющим общую архитектуру, надёжность и масштабируемость создаваемого веб-сервиса. На данном этапе осуществляется планирование внутренних компонентов системы, определение взаимосвязей между её частями и выбор способов организации данных. Качественно проработанная структура позволяет минимизировать риски возникновения ошибок при реализации и закладывает основу для дальнейшего расширения функционала. В рамках текущего проекта данный процесс ориентирован на создание высокопроизводительного решения, способного эффективно обрабатывать действия пользователей в реальном времени.

Основной упор при проектировании делается на разделение логики приложения в соответствии с современными архитектурными шаблонами. Это позволяет изолировать интерфейс пользователя от серверных вычислений и процедур прямого взаимодействия с реляционной базой данных. Серверная часть проектируется как модуль, отвечающий за обработку входящих сетевых пакетов, авторизацию участников, создание игровых комнат и контроль над соблюдением правил карточных сессий [2][4]. Клиентская часть, в свою очередь, проектируется как легковесный модуль, главной задачей которого является корректное отображение полученных от сервера данных и отправка действий пользователя [3].

Важным аспектом проектирования является обеспечение отказоустойчивости и высокой скорости отклика системы при одновременной активности множества независимых лобби. Архитектурные решения должны гарантировать, что критические сбои или задержки в одной игровой комнате никак не повлияют на стабильность работы остальных активных сессий. Для этого внедряются механизмы изоляции потоков данных и распределения вычислительных ресурсов сервера. Проектируемая логика обработки событий строится на принципах асинхронности, что критически важно для аналогичных систем, функционирующих в условиях постоянного входящего трафика.

Спроектированная архитектура служит технологическим базисом для последующего перехода к этапу непосредственной программной реализации всех модулей веб-сервиса. В последующих разделах подробно рассматриваются вопросы построения внутренней структуры серверных и клиентских модулей, а также механизмы организации дуплексного обмена сообщениями. Разработанные проектные решения позволяют создать логичную, понятную и гибкую программную систему, полностью удовлетворяющую всем ранее поставленным техническим и функциональным требованиям к разрабатываемому продукту.

2.1 Проектирование структуры программы

Архитектура разрабатываемого игрового веб-приложения построена на принципах модульного проектирования и структурно разделена на два фундаментальных блока: сервер и клиент, исполняемый в браузере. Подобное разделение позволяет минимизировать сетевые издержки, обеспечить высокую производительность под нагрузкой и разграничить зоны ответственности между интерфейсом и ядром бизнес-логики. Внутренняя структура сервера организована по слоистой архитектурной парадигме, где каждый уровень выполняет определённый набор функций и предоставляет интерфейсы для смежных компонентов, изолируя сетевую логику от прямого взаимодействия с базой данных.

Первый базовый уровень серверной части представлен слоем взаимодействия, функционирующим на основе сетевого протокола *WebSocket* для поддержки постоянных полнодуплексных соединений [1]. На этапе подключения система верифицирует сессии пользователей, привязывает сетевые сокеты к конкретным профилям и распределяет активных игроков по изолированным виртуальным комнатам. Этот слой организует мгновенную трансляцию асинхронных событий всем участникам соревновательного лобби. Потокобезопасная обработка сокетов на сервере реализуется средствами легковесных горутин, что позволяет эффективно обслуживать множество одновременных сессий.

Центральное место в серверной архитектуре занимает слой изолированной бизнес-логики, вычислительным ядром которого служит специализированный математический модуль карточной игры. Этот компонент инкапсулирует в себе алгоритмы генерации и тасования виртуальных колод, функции автоматического расчёта и валидации достоинства игровых комбинаций, а также серверные алгоритмы контроля игровых комнат. Данная логика гарантирует безопасное и последовательное переключение фаз матча: инициализацию игры, раздачу карт, обработку поочерёдных ходов и финальное определение победителя, что полностью исключает возможность мошенничества.

За надёжное хранение информации отвечает слой доступа к СУБД *PostgreSQL* [2]. Персистентное хранение учётных записей, профилей пользователей и статистики матчей делегировано базе данных, а динамическое состояние активных столов обрабатывается в оперативной памяти сервера. Клиентская часть спроектирована как автономное приложение, отвечающее за поддержание соединения, обработку *JSON*-пакетов и визуализацию интерфейса при помощи *CSS*-анимаций [3][5]. Сервер выступает координатором, верифицирующим команды и фиксирующим результаты игровых сессий.

2.2 Проектирование системы сетевого взаимодействия

Сетевое взаимодействие является наиболее критичным узлом системы. Традиционная модель *HTTP*, основанная на одиночных запросах и ответах, непригодна для фазы активного игрового процесса, поскольку сервер не обладает механизмом самостоятельной инициации отправки данных клиенту в момент совершения хода оппонентом. Для решения данной проблемы спроектирована асинхронная событийно-ориентированная архитектура на базе полнодуплексного протокола *WebSocket* [1]. Подобное решение позволяет установить одно постоянное соединение, через которое пакеты могут непрерывно передаваться в обоих направлениях между клиентами и серверной частью.

Взаимодействие начинается с этапа установления связи, выполняемого посредством стандартной процедуры согласования соединения. При переходе пользователя на страницу лобби клиентская часть отправляет первичный *HTTP*-запрос, содержащий обязательный заголовок о переключении протокола. Сервер на языке Go перехватывает данный запрос, извлекает сессионные данные для верификации пользователя через *PostgreSQL* и подтверждает переключение на *WebSocket* [1][2]. Установленному сокету присваивается дескриптор, после чего сервер выделяет изолированную горутину для прослушивания входящего потока данных и привязывает соединение к профилю [6].

Исходящий поток данных от клиента к серверу строго регламентирован и обрабатывается через фиксированный набор структурированных *JSON*-пакетов, содержащих тип события и полезную нагрузку [5]. Процесс подключения к матчу инициируется отправкой пакета авторизации в комнату, в рамках которого передаётся уникальный идентификатор лобби для распределения участников. Непосредственное управление процессом карточной дуэли осуществляется посредством трансляции команд о ходах игроков, отправляемых при наступлении их очереди. Клиентские обработчики преобразуют действия пользователя в интерфейсе в строго типизированные сетевые запросы.

Входящий поток данных от сервера к клиенту инкапсулирует в себе пакеты асинхронного управления интерфейсом, рассылаемые участникам дуэли [4]. После успешного сбора двух игроков в лобби сервер генерирует и транслирует пакет о начале матча, запускающий раздачу карт, а затем передаёт сигналы о переходе права хода. В процессе игры клиенты получают мгновенные обновления, фиксирующие изменения на столе после действий оппонента, а по завершении сессии сервер рассылает отчёт с результатами матча. Спроектированный протокол транслирует только дельта-обновления, что гарантирует отзывчивость интерфейса.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Логическая структура файлов разрабатываемого проекта организована в виде иерархии чётко обозначенных каталогов, что значительно облегчает общую навигацию, ускоряет процесс поиска необходимых компонентов и закладывает основу для масштабирования системы. Каждая группа элементов вынесена в отдельное изолированное пространство, что полностью исключает конфликты между внешним интерфейсом и защищённым контуром бизнес-логики. Подробная схема сформированного дерева каталогов программного комплекса со всеми ключевыми модулями и конфигурационными файлами представлена ниже.

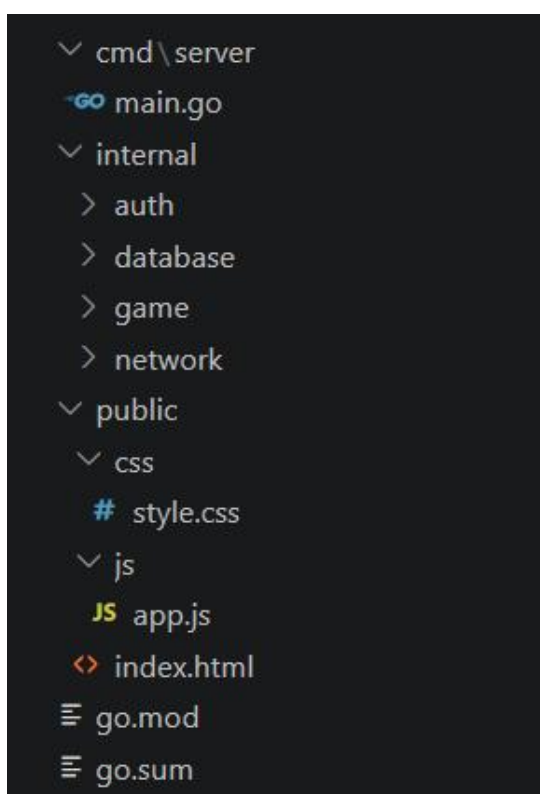


Рисунок 3.1 – Файловая структура

Приложение запускается из файла `main.go`, а базовые конфигурации зависимостей зафиксированы в файлах `go.mod` и `go.sum`. Архитектура внутренних модулей изолирована внутри директории `internal`, где пакет `auth` отвечает за сессии, `database` обеспечивает работу с *PostgreSQL*, `game` реализует математику карточной дуэли, а `network` отвечает за сетевое взаимодействие по протоколу *WebSocket* [1][2]. Фронтенд и все статические ресурсы вынесены в директорию `public`, где файл `index.html` создаёт каркас интерфейса, файл `style.css` отвечает за визуальное оформление и плавные анимации, а скрипт `app.js` управляет клиентом [3][4][7].

3.1 Описание разработанных модулей

Разработанное веб-приложение построено на основе модульной архитектуры, обеспечивающей строгое разделение ответственности между компонентами клиентской и серверной частей. Созданное программное средство состоит из нескольких функционально взаимосвязанных логических блоков, каждый из которых надёжно инкапсулирует определённый набор специализированных функций. Подобная организация исходного кода существенно упрощает процессы отладки и последующего масштабирования всей системы, позволяя модифицировать отдельные независимые подсистемы без риска нарушения общей целостности и стабильности функционирования всего программного комплекса.

Модуль доступа к данным и управления сессиями объединяет пакеты *database* и *auth* для организации хранения информации и контроля текущих пользовательских состояний. Серверная часть комплекса использует реляционную систему управления базами данных *PostgreSQL*, которая отвечает за хранение учётных записей игроков, параметров профилей и истории всех проведённых карточных партий [2]. Специализированный компонент авторизации постоянно контролирует активные сессии подключённых пользователей, выполняет верификацию клиентов при входе в систему и обеспечивает безопасный санкционированный доступ к контексту главного игрового лобби.

Модуль бизнес-логики координируется в рамках серверного пакета *game* и содержит в себе ключевые алгоритмы, реализующие математические правила соревновательной карточной дуэли. Этот архитектурный слой полностью абстрагирован от используемых сетевых протоколов и занимается исключительно внутренними расчётами игровых этапов. В непосредственные обязанности данного модуля входит генерация стандартной колоды карт, их случайное перемешивание по внутренним механизмам, раздача участникам дуэли, точное определение собранных покерных комбинаций и фиксация победы с последующей автоматической передачей итоговых результатов в модуль базы данных [8].

Модуль сетевого взаимодействия и пользовательского интерфейса координирует обмен сообщениями и обеспечивает наглядное визуальное представление всего игрового процесса. Серверный пакет *network* управляет открытыми соединениями по протоколу *WebSocket*, обеспечивая мгновенную двустороннюю доставку *JSON*-пакетов между сервером и активными пользователями [1][5]. На стороне клиента скрипт *app.js* обрабатывает входящие сетевые события и динамически перестраивает структуру разметки страницы *index.html*, а файл стилей *style.css* отвечает за минималистичное оформление лобби в полном соответствии с дизайном [3][4].

3.2 Описание сетевого модуля

Сетевой модуль выступает центральным коммуникационным ядром создаваемого программного комплекса, обеспечивая полнодуплексный обмен данными между пользователями в режиме реального времени. Его логика разделена на два ключевых компонента, к которым относятся серверный пакет *network* и клиентский скрипт *app.js*. Взаимодействие начинается с инициализации соединения на стороне веб-браузера, где вызов функции подключения устанавливает стандарт *WebSocket* в качестве единственного транспортного механизма на базе протокола *TCP* [1][9]. Это позволяет исключить резервные запросы и минимизировать задержки при передаче *JSON*-пакетов [5].

Архитектура клиентской части полностью выстроена на асинхронной обработке входящих сообщений, поступающих от серверной части. При первичном подключении или запросе синхронизации клиент получает сетевой пакет, содержащий полный снимок текущего состояния игровой комнаты. Обработчик данного события кэширует информацию в локальные переменные браузера и запускает метод обновления интерфейса [4]. Этот алгоритм детально анализирует текущую фазу дуэли, после чего последовательно перестраивает структуру разметки, отображает карты участников и обновляет лобби.

Динамика активного игрового процесса обеспечивается постоянной обработкой атомарных сетевых событий, которые транслируются сервером после каждого действия участников покерной дуэли. Информационный пакет содержит идентификатор совершившего ход пользователя, обновлённые массивы карт на столе и маркер активного игрока, что позволяет клиентскому скрипту выполнять точечное обновление элементов стола. Передача права хода обновляет глобальный контекст приложения и активирует необходимые кнопки управления. Метод строго проверяет состояние стола, поэтому элементы интерфейса становятся доступными только при строгом совпадении очереди хода пользователя.

Завершение карточной дуэли координируется через обработку финального серверного пакета, содержащего итоговые результаты завершённой партии. При его получении клиентская часть блокирует элементы управления ходами, наглядно раскрывает карты оппонента и выводит подробную сводку игры в интерфейс лобби. Алгоритм дифференцирует исходы раунда, определяя победу или поражение на основе собранных покерных комбинаций, и фиксирует изменения в базе данных *PostgreSQL* [2]. После демонстрации результатов сервер автоматически переводит пользовательский интерфейс в режим готовности, подготавливая интерфейс приложения к запуску новой игровой сессии.

4 ТЕСТИРОВАНИЕ

Тестирование созданного веб-приложения проводилось с целью подтверждения корректности функционирования серверной логики, контроля карточных правил дуэли, оценки стабильности *WebSocket*-соединения и защиты от подмены данных со стороны клиента. Все тесты были напрямую сфокусированы на поиске критических ошибок, изучении поведения алгоритмов при нестандартных действиях пользователей и подтверждении полной работоспособности софта. Такой подход помог быстро обнаружить и устранить скрытые ошибки в исходном коде, что позволило запустить серверную часть без сбоев и обеспечить стабильную синхронизацию всех текущих сессий.

В рамках контроля модуля управления пользователями оценивалась корректность обработки запросов на регистрацию и авторизацию игроков. При передаче валидных учётных данных сервер успешно осуществляет хэширование пароля, создаёт новую запись в базе данных *PostgreSQL* и возвращает статус успешной обработки, открывая активную сессию в рамках пакета *auth* [2][10]. В случае попытки регистрации с использованием уже существующего в системе псевдонима сервер через пакет *database* сразу обнаруживает совпадение и возвращает ошибку, что полностью исключает дублирование профилей.

Тестирование подсистемы игровых комнат подтвердило правильность заложенных алгоритмов автоматического распределения участников покерной дуэли. При создании нового стола с заданными параметрами система выделяет память под новую игровую сессию, генерирует уникальный идентификатор для подключения и автоматически регистрирует создателя сессии внутри новой комнаты. Попытки несанкционированного входа в закрытую сессию или отправка некорректных параметров на этапе анализа успешно блокируются серверной частью, которая прерывает дальнейшую обработку запроса, после чего пользователю отказывается в доступе без инициализации сетевого сокета [1].

Основополагающим этапом стала верификация пакета *game*, которая включала детальную оценку пользовательских сценариев и модульное тестирование чистых математических функций. В ходе проверок детально анализировалась работа логики при возникновении коллизий, передаче пустых ходов, превышении лимитов ставок и обработке других критических условий. Автоматизированные тесты подтвердили, что разработанная серверная система корректно распознает все возможные карточные сочетания, правильно определяет их старшинство при сравнении рук участников дуэли на этапе вскрытия и передаёт полученные результаты в сетевой модуль *network* для формирования *JSON*-пакетов [5].

Динамика активного игрового процесса проверялась на соответствие правилам переходов между фазами ожидания готовности участников и непосредственной дуэли. Алгоритм контроля очерёдности ходов исключает возможность возникновения состояния гонки данных, так как любые попытки клиента отправить пакет вне очереди немедленно отклоняются серверной частью. Идентификатор активного игрока зафиксирован в состоянии комнаты на стороне бэкенда, поэтому несвоевременные команды от сторонних пользователей игнорируются системой и не нарушают игровой процесс. При этом только валидное действие от активного игрока успешно обрабатывается сервером, обновляет глобальный таймер сессии, инициирует широковещательную рассылку статуса стола и исключает рассинхронизацию между клиентами.

Помимо стандартных функциональных проверок, было проведено тестирование разработанных механизмов обработки внештатных ситуаций, в частности устойчивости системы к внезапным обрывам сетевого соединения. При симуляции потери сети клиентом серверный модуль *network* корректно перехватывал событие закрытия *WebSocket*-канала по истечении таймаута. Система автоматически завершала ход отключившегося пользователя или присуждала техническое поражение, что полностью исключало зависание виртуальной комнаты и блокировку игрового процесса для оставшегося активного участника дуэли [1].

Тесты надёжности подтвердили согласованность и сохранность данных при экстренном прерывании пользовательской сессии. Все итоговые изменения параметров, рассчитанные пакетом *game* в оперативной памяти, синхронизировались с хранилищем *PostgreSQL* через пакет *database* [2]. Проверка показала, что при принудительном закрытии вкладки браузера в момент фиксации результатов информация о завершённой партии, текущем балансе и очках игроков сохраняется в базе данных корректно, а все незавершённые операции автоматически отменяются, что полностью исключает появление висящих записей и нарушение логической целостности таблиц даже при пиковых нагрузках.

При контроле устойчивости к сетевым угрозам оценивалась защищённость серверной логики от подмены пакетов. Попытки модификации локальных переменных клиентского интерфейса в браузере приводили только к нарушениям отображения элементов интерфейса только на устройстве нарушителя. При фиксации фактического игрового действия с изменёнными параметрами серверная часть сразу блокировала операцию при проверке структуры входящего *JSON*-пакета, опираясь на идентификатор сокета текущей сессии и актуальное состояние стола, что полностью подтверждает строгость заложенных правил обработки данных и гарантирует защиту от инъекций со стороны внешних нарушителей [5].

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Настоящее руководство позволяет ознакомиться с интерфейсом веб-приложения и анализом сетевых пакетов. Фиксация трафика связывает сетевые запросы пользователя с работой серверной части. Взаимодействие с системой начинается с перехода по адресу сервера, после чего в браузере открывается интерфейс авторизации [4].

В момент обращения к серверу в сети фиксируется стандартная последовательность запросов для загрузки компонентов приложения. Первичный запрос возвращает структуру страницы, после чего браузер скачивает файлы стилей и скрипты [3][7]. Сервер возвращает статус обработки, завершая визуализацию.

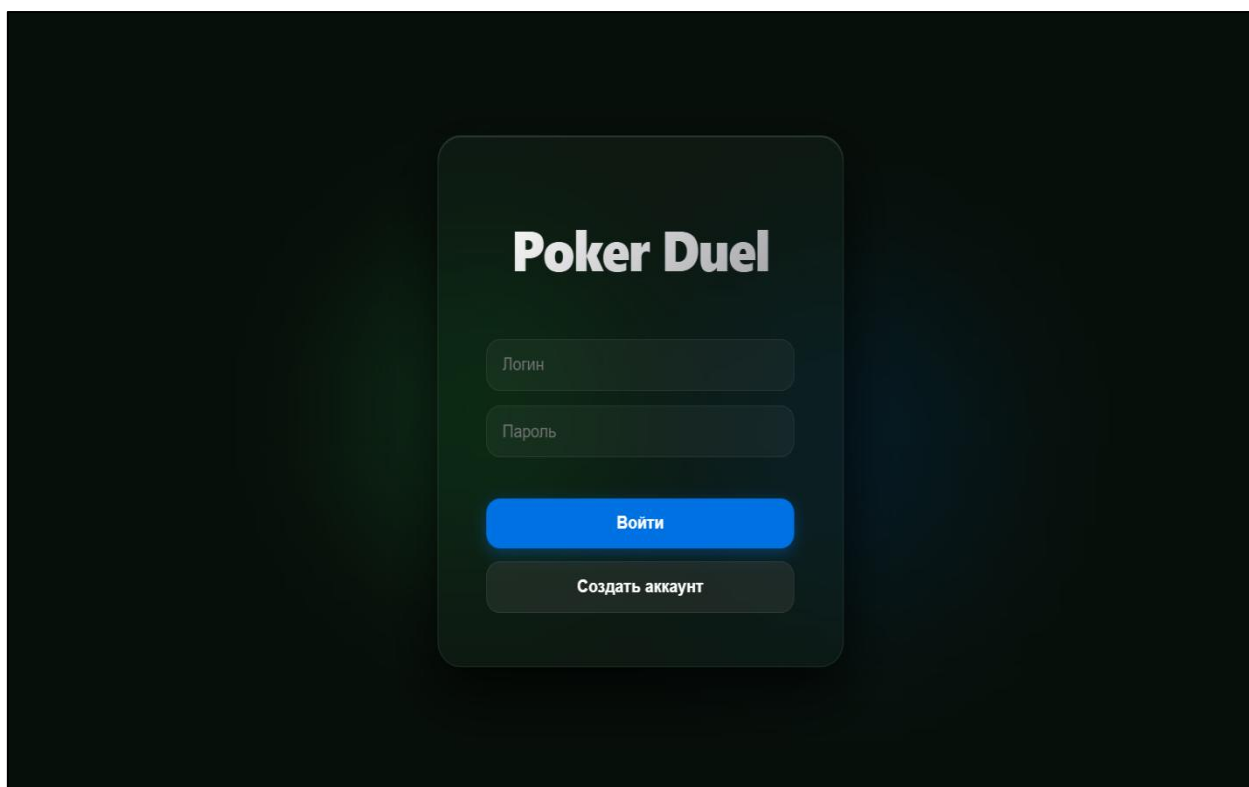


Рисунок 5.1 – Окно авторизации пользователя

```
GET / HTTP/1.1
HTTP/1.1 200 OK (text/html)
GET /css/style.css HTTP/1.1
GET /js/app.js HTTP/1.1
HTTP/1.1 200 OK (text/css)[Illegal Segments]
HTTP/1.1 200 OK (text/javascript)
```

Рисунок 5.2 – Запрос на получение окна авторизации

Для создания новой учётной записи в приложении предусмотрено специализированное окно аутентификации с полями логина и пароля [4]. При корректном заполнении данных и отправке формы в сети фиксируется *POST*-запрос, который обрабатывается сервером [9]. В случае правильного выполнения операции сервер возвращает статус 200 *OK*, подтверждая создание нового пользователя.

Если пользователь пытается зарегистрировать аккаунт под именем, которое уже присутствует в системе, интерфейс выводит предупреждение о занятом логине. В сетевом трафике при этом фиксируется аналогичный *POST*-запрос, на который сервер отвечает ошибкой 409 *Conflict*. Такая валидация на стороне сервера исключает создание дублирующихся учётных записей в базе данных [2].

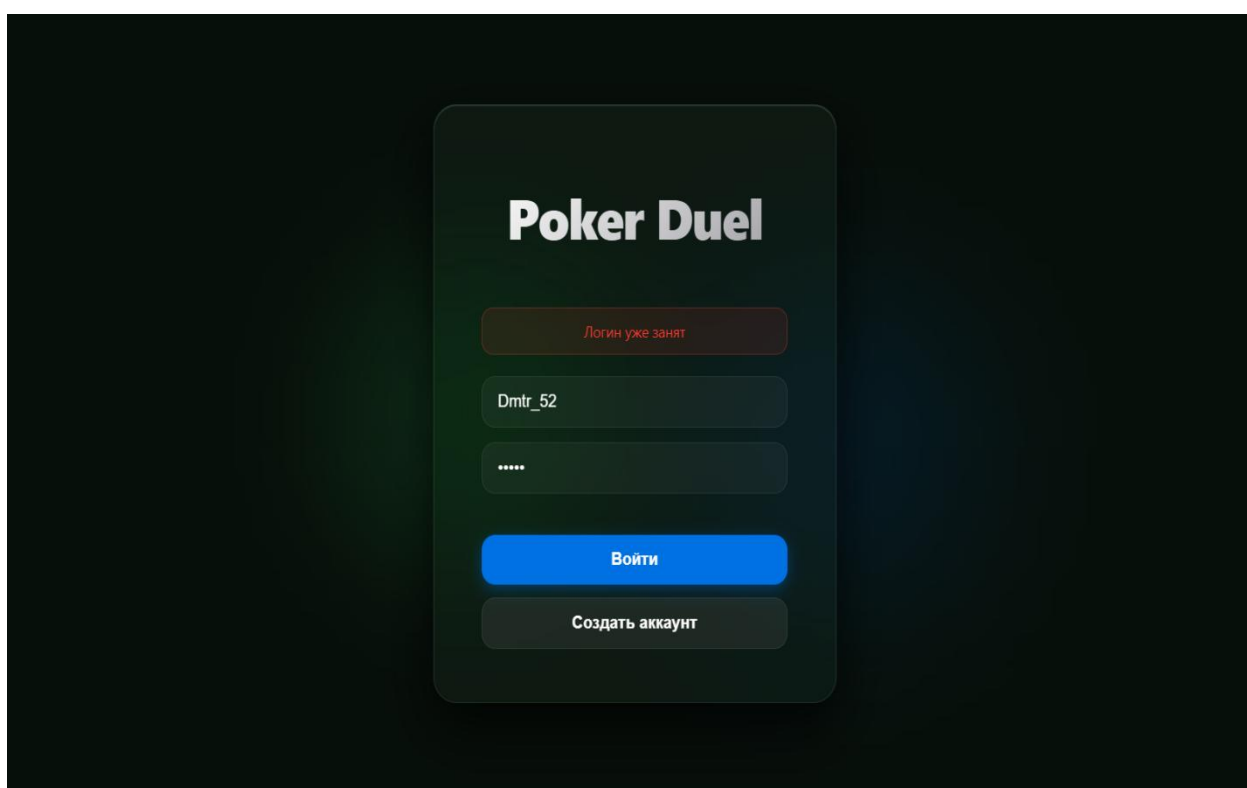


Рисунок 5.3 – Окно с сообщением ошибки регистрации

```
POST /api/auth/register HTTP/1.1 (application/x-www-form-urlencoded)
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.4 – Запрос на успешную регистрацию аккаунта

```
POST /api/auth/register HTTP/1.1 (application/x-www-form-urlencoded)
HTTP/1.1 409 Conflict , JSON (application/json)
```

Рисунок 5.5 – Запрос на ошибочную регистрацию аккаунта

Для создания новой учётной записи в приложении предусмотрено специализированное окно аутентификации с полями логина и пароля [4]. При корректном заполнении данных и отправке формы в сети фиксируется *POST*-запрос, который обрабатывается сервером [9]. Если учётные данные полностью совпадают с записями в базе, сервер возвращает статус 200 *OK*, завершая процесс входа [2].

Если пользователь указывает неверную комбинацию имени или пароля, система блокирует попытку аутентификации. В сетевом трафике при этом фиксируется аналогичный *POST*-запрос, на который сервер отвечает ошибкой 401 *Unauthorized*. Встроенная валидация на стороне сервера защищает пользовательские профили от несанкционированного доступа методом простого подбора [10].

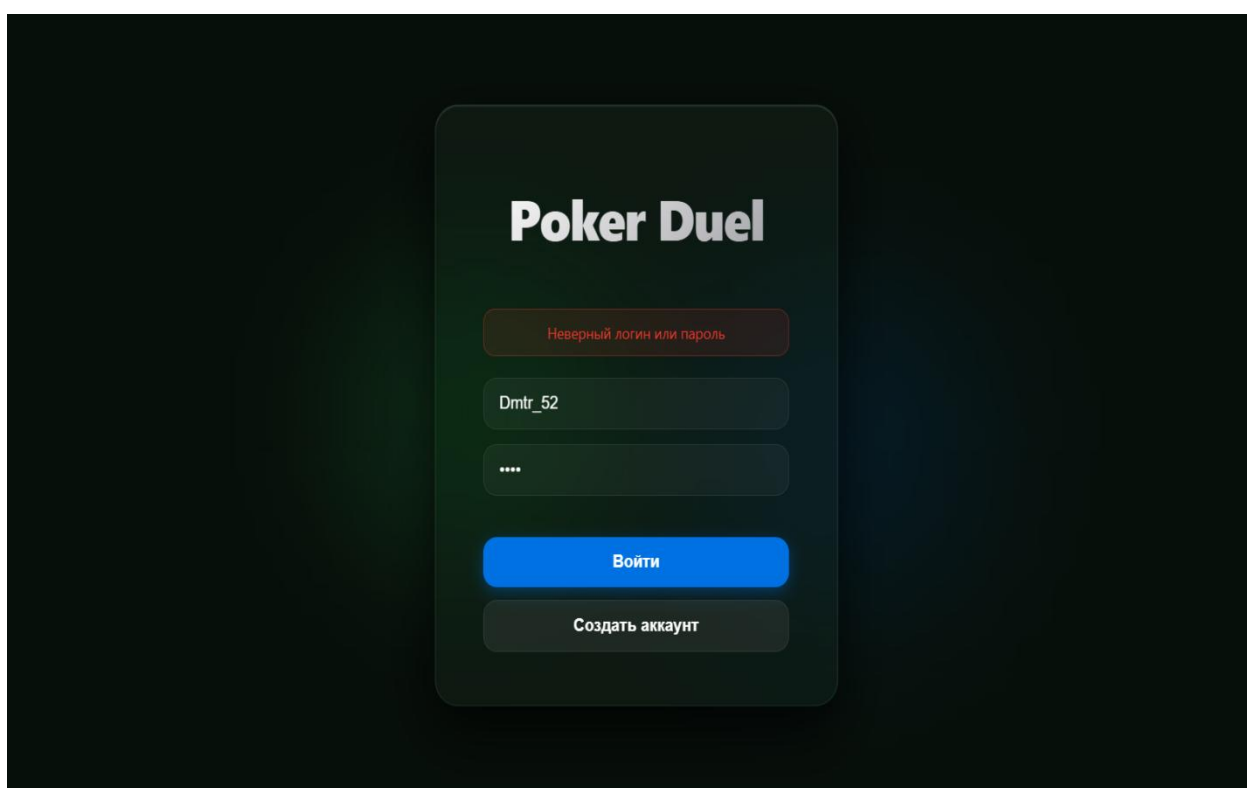


Рисунок 5.6 – Окно с сообщением ошибки авторизации

```
POST /api/auth/login HTTP/1.1 (application/x-www-form-urlencoded)
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.7 – Запрос на успешную авторизацию аккаунта

```
POST /api/auth/login HTTP/1.1 (application/x-www-form-urlencoded)
HTTP/1.1 401 Unauthorized , JSON (application/json)
```

Рисунок 5.8 – Запрос на ошибочную авторизацию аккаунта

После успешного прохождения авторизации пользователю открывается главное меню веб-приложения. В верхней части экрана располагается общая информационная панель, отображающая текущий баланс игровых фишек, количество заработанных трофеев и кнопку доступа к статистике профиля [4]. Основную площадь страницы занимают четыре интерактивные плитки для выбора конкретного режима игры, которые разделены по внутренней логике взаимодействия с сервером [3]. Доступные пользователю режимы арена, спины и турнир запускают стандартную сессию одиночного противостояния, где в качестве соперника выступает обычный бот. Единственным режимом для онлайн-взаимодействия между реальными участниками является плитка друзья, позволяющая самостоятельно создавать закрытые сессии по уникальному коду [1].

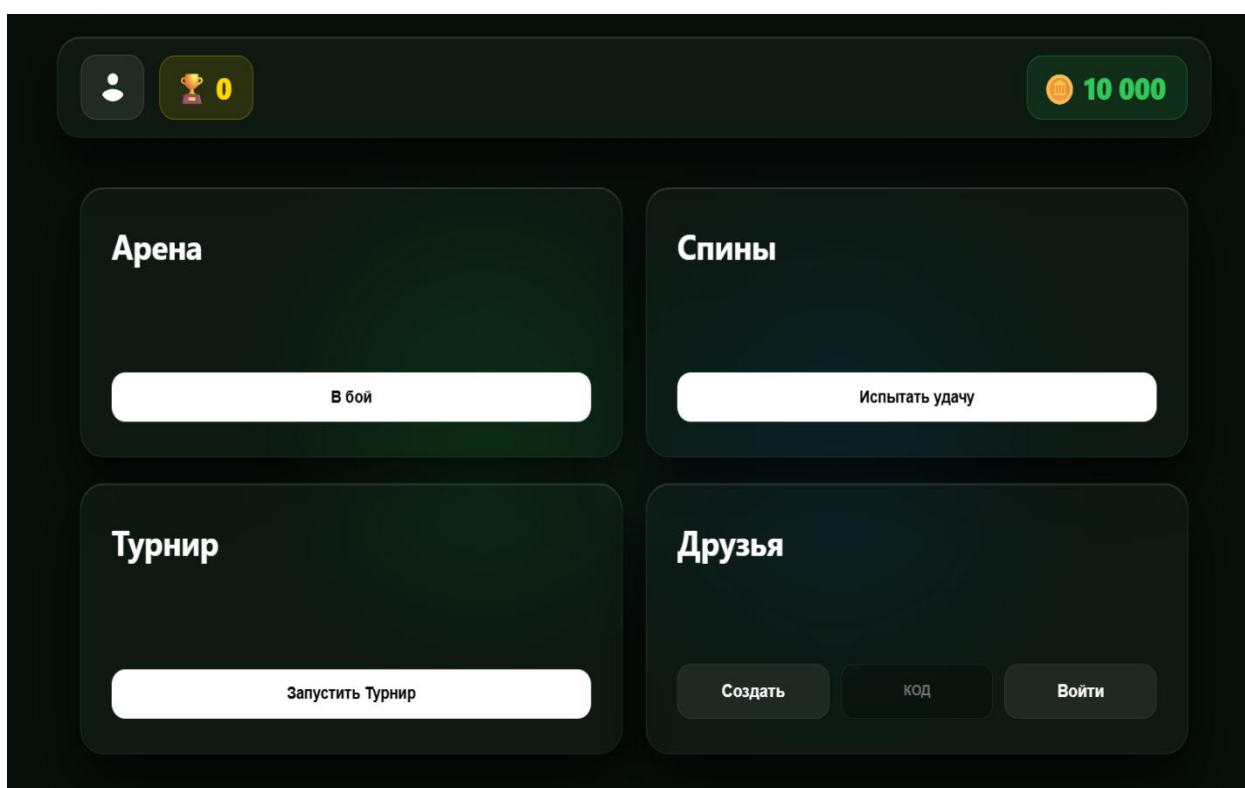


Рисунок 5.9 – Главное меню приложения

Выбор любого из режимов отправляет запросы на сервер для инициализации игровой сессии и проверки баланса фишек. При нажатии на кнопки запуска одиночных матчей сервер генерирует виртуальный игровой стол и автоматически подключает к нему бота. В режиме взаимодействия с друзьями логика работы сервера усложняется, так как система переходит в режим ожидания второго подключения [6]. Сервер контролирует состояние сессии на каждом этапе игрового процесса, исключая возможность участия одного пользователя в нескольких матчах одновременно [2].

После запуска любого режима пользователю открывается интерфейс игрового стола для проведения матча. В центральной части экрана отображаются текущие карты игрока, рубашки карт соперника, а также актуальный размер банка сформированного в ходе раздачи. Дополнительно на столе выводятся текущие балансы фишек обоих участников для постоянного визуального контроля за ходом проведения игровой сессии [3]. Справа от игрового поля располагается отдельная информационная панель, которая в режиме реального времени показывает текущую собранную комбинацию пользователя и расчётную вероятность его победы [4]. В самом низу экрана находятся интерактивные кнопки управления действиями, позволяющие сбрасывать карты, делать фиксированные ставки, уравнивать их или сразу идти ва-банк.

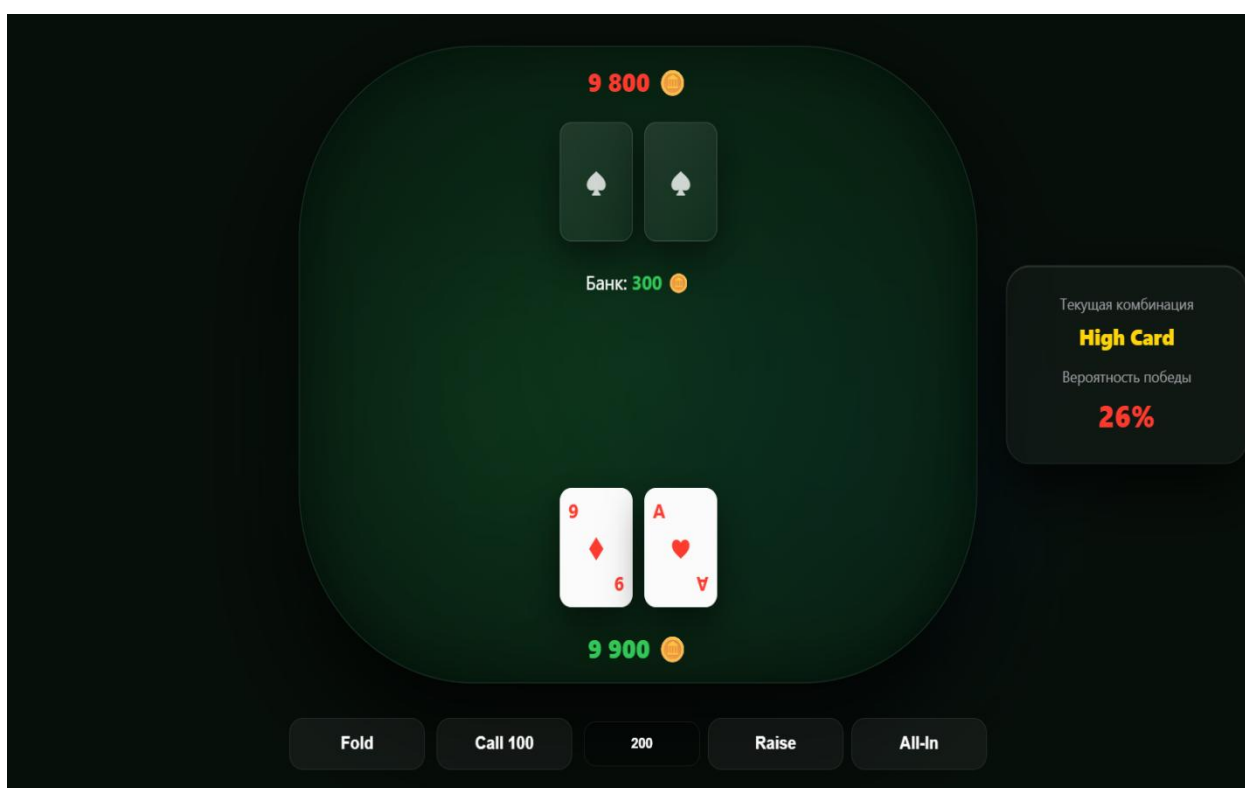


Рисунок 5.10 – Игровой стол приложения

Выбор любого действия на панели управления отправляет запросы на сервер для его выполнения и проверки баланса фишек [1]. При совершении хода в одиночном матче сервер обрабатывает это действие и выполняет ответный ход за бота. В режиме игры с друзьями логика работы сервера меняется, так как система переходит в режим ожидания действия второго игрока [5]. Процесс подсчёта комбинаций выполняется на стороне сервера для обеспечения безопасности [8]. Сервер контролирует состояние сессии на каждом этапе, исключая возможность невалидных ходов [6].

При нажатии на кнопку профиля в главном меню пользователю открывается отдельное окно со сводной статистикой его учётной записи. В верхней части данного окна последовательно отображаются три информационных блока, содержащие общее количество сыгранных матчей, процент выигранных игр и размер наибольшего единовременного выигрыша. Ниже располагается динамический список последних завершённых матчей с текстовым указанием игрового режима и числовым маркером количества полученных или потерянных фишек с цветовой индикацией [3]. В самой нижней части интерфейса находится кнопка управления, которая позволяет пользователю закрыть текущее окно статистики и вернуться на главную страницу веб-приложения для продолжения взаимодействия с игровыми режимами [4].

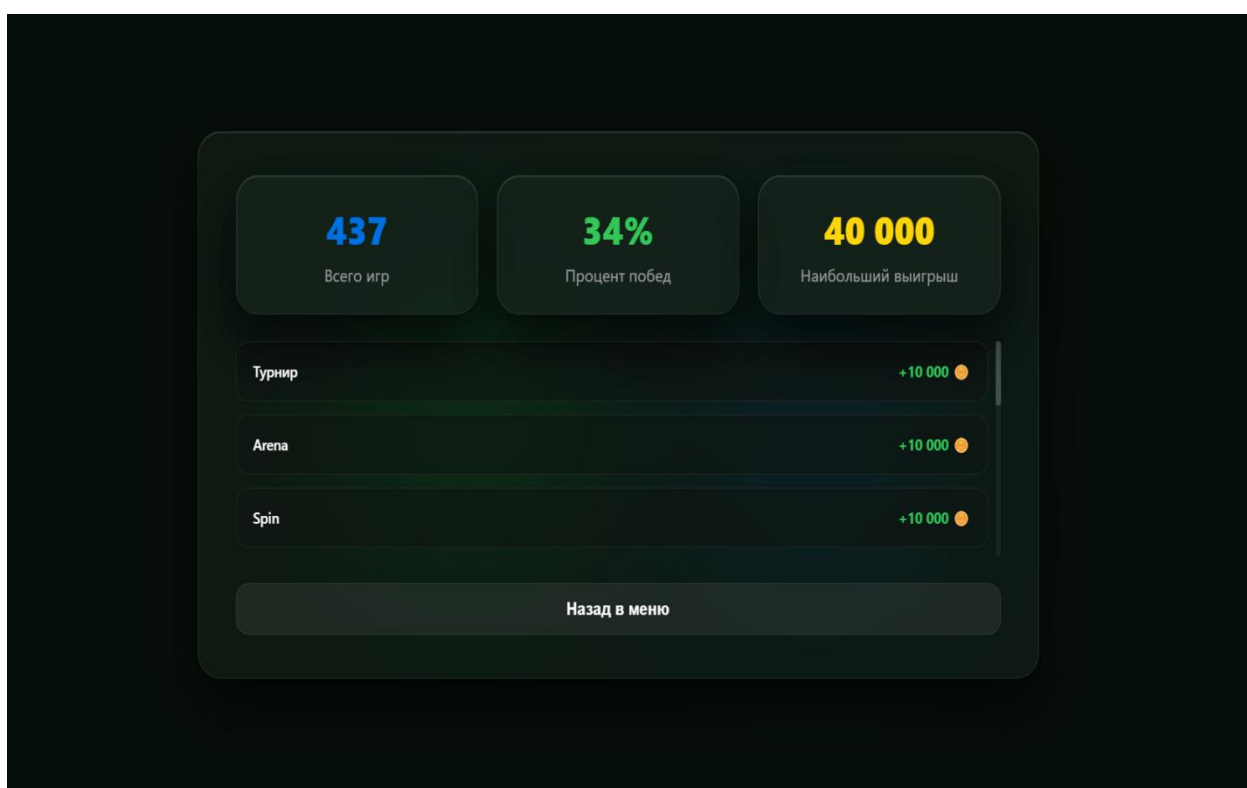


Рисунок 5.11 – Окно статистики профиля

Переход в данное окно активирует отправку запроса на сервер для извлечения актуальных данных пользователя из базы данных [9]. Сервер обрабатывает этот запрос, проверяет идентификатор сессии и собирает информацию обо всех сыгранных пользователем матчах [6]. Вся процедура расчёта процента побед и фильтрации истории игр выполняется на стороне сервера [2]. Сервер контролирует процесс передачи сформированных пакетов данных в пользовательский интерфейс, исключая возможность отображения невалидной информации [5].

Инициирование сессии первого пользователя начинается с отправки запроса на установление постоянного *WebSocket*-соединения. В трафике фиксируется процедура обновления протокола *HTTP* и переход канала в режим обмена текстовыми фреймами [1].

```
GET /ws?user=11111 HTTP/1.1  
HTTP/1.1 101 Switching Protocols  
WebSocket Text [FIN] [MASKED]
```

Рисунок 5.12 – Переход на протокол WebSocket

Для генерации игровой комнаты клиент отправляет *JSON*-пакет через установленный дуплексный канал [5]. В теле запроса передаётся тип действия для создания сессии, а также идентификатор, используемый как код подключения участника [8].

```
{"action": "create_friend", "code": "JUKN"}
```

Рисунок 5.13 – Структура пакета при создании комнаты

Взаимодействие со стороны второго пользователя начинается с отправки запроса на переключение протокола для инициализации *WebSocket*-канала [1]. После подтверждения соединения сервером клиент получает возможность отправки пакетов [9].

```
GET /ws?user=22222 HTTP/1.1  
HTTP/1.1 101 Switching Protocols  
WebSocket Text [FIN] [MASKED]
```

Рисунок 5.14 – Переход на протокол WebSocket

Для авторизации в комнате второй клиент транслирует в сеть целевой фрейм данных в формате *JSON* [5]. В пакете содержится запрос на подключение к сессии, а также передаётся буквенный код, введённый пользователем в интерфейсе [4].

```
{"action": "join_friend", "code": "JUKN"}
```

Рисунок 5.15 – Структура пакета при входе в комнату

При отправке пользователем команды *check* сервер проверяет, что его ставка уже равна максимальной ставке за столом. Затем сервер увеличивает счётчик сделавших ход игроков и передаёт действие следующему участнику, вызывая функцию очереди [6].

```
WebSocket Text [FIN] [MASKED]  
WebSocket Text [FIN]  
WebSocket Text [FIN]
```

Рисунок 5.16 – Сетевые пакеты операции «check»

После фиксации хода сервер отправляет пакеты с обновлённым состоянием игры на оба аккаунта. На стороне клиента визуально меняется очередь хода, а в параметрах последнего действия сохраняется текстовый статус проверки [3][8].

```
{"action": "check"}
```

Рисунок 5.17 – Структура пакета операции «check»

При отправке пользователем команды *call* сервер рассчитывает сумму для уравнивания как разность между максимальной и ставкой игрока [5]. Если у пользователя не хватает фишек в стеке, система активирует статус ва-банка и забирает весь баланс [2].

```
WebSocket Text [FIN] [MASKED]  
WebSocket Text [FIN]  
WebSocket Text [FIN]
```

Рисунок 5.19 – Сетевые пакеты операции «call»

Все полученные фишки сервер добавляет в банк, приравнивает ставку игрока к максимальной и увеличивает счётчик сделавших ход игроков [1]. На экране клиента отображается уменьшение личных фишек, увеличение банка и статус ответа [4].

```
{"action": "call"}
```

Рисунок 5.20 – Структура пакета операции «call»

При отправке пользователем команды *raise* сервер устанавливает указанную сумму в качестве новой максимальной ставки стола и рассчитывает размер доплаты. Если фишек для совершения рейза не хватает, активируется статус ва-банка [5].

```
WebSocket Text [FIN] [MASKED]  
WebSocket Text [FIN]  
WebSocket Text [FIN]
```

Рисунок 5.21 – Сетевые пакеты операции «raise»

Сервер сбрасывает счётчик сделавших ход игроков до единицы, чтобы соперник был обязан сделать ответное действие, после чего вызывает функцию очереди. На экране клиента уменьшаются фишки и увеличивается общий банк стола [3].

```
{"action": "raise", "amount": 100}
```

Рисунок 5.22 – Структура пакета операции «raise»

При отправке пользователем команды *fold* сервер присваивает игроку статус паса, полностью исключая его из текущей раздачи. Сервер сразу определяет соперника победителем, начисляет ему весь банк стола и меняет статус текущей игровой фазы [1].

```
WebSocket Text [FIN] [MASKED]  
WebSocket Text [FIN]  
WebSocket Text [FIN]
```

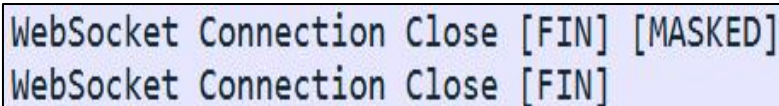
Рисунок 5.23 – Сетевые пакеты операции «fold»

Если проигравший участник остался с нулевым балансом, система отправляет пакет о полном окончании игры [8]. Если фишки есть, то через две секунды запускается новый раунд. На экране клиента сбросившего карты фиксируется пас [4].

```
{"action": "fold"}
```

Рисунок 5.24 – Структура пакета операции «fold»

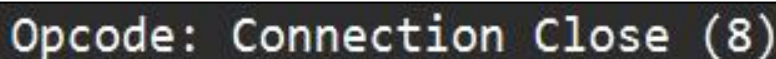
При выходе первого участника из игровой комнаты приложение отправляет специальный закрывающий фрейм для прекращения сессии связи. Сервер принимает этот запрос, фиксирует намерение пользователя покинуть стол и отправляет зеркальный ответ для корректного завершения процедуры рукопожатия [1].



WebSocket Connection Close [FIN] [MASKED]
WebSocket Connection Close [FIN]

Рисунок 5.25 – Сетевые пакеты при закрытии соединения

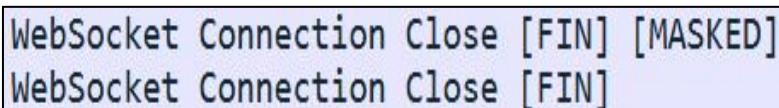
Внутри отправленного пакета также содержится код закрытия, который сигнализирует серверной части о разрыве соединения со стороны игрока. После этого обмена транспортный протокол сразу приступает к закрытию сетевого канала [7].



Opcode: Connection Close (8)

Рисунок 5.26 – Структура пакета с кодом закрытия сессии

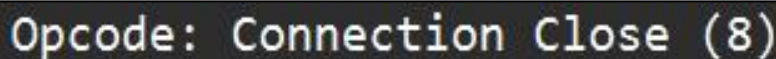
При выходе второго участника из игровой комнаты запускается аналогичный процесс завершения работы активного сетевого соединения. Клиентское приложение отправляет финальный закрывающий фрейм, полностью прекращая дальнейший обмен любыми игровыми данными с сервером [1].



WebSocket Connection Close [FIN] [MASKED]
WebSocket Connection Close [FIN]

Рисунок 5.27 – Сетевые пакеты при закрытии соединения

Внутри отправленного пакета также содержится код закрытия, который сигнализирует серверной части о разрыве соединения со стороны игрока. После этого обмена транспортный протокол сразу приступает к закрытию сетевого канала [5].



Opcode: Connection Close (8)

Рисунок 5.28 – Структура пакета с кодом закрытия сессии

ЗАКЛЮЧЕНИЕ

В процессе выполнения проекта была успешно спроектирована, реализована и протестирована масштабируемая многопользовательская программная система для управления игровым процессом. Актуальность проведённой работы обусловлена растущим спросом на веб-приложения, требующие мгновенного отклика и надёжной синхронизаций состояний между множеством клиентов. Использование выбранных решений позволило реализовать надёжную логику сервера, способную эффективно обрабатывать входящие потоки данных и обслуживать запросы пользователей с минимальной задержкой.

В ходе аналитического этапа был проведён детальный анализ предметной области и выявлены ключевые проблемы организации сетевого взаимодействия в многопользовательской среде. Для обхода стандартных ограничений классических протоколов была выбрана асинхронная парадигма обмена данными, которая снижает нагрузку на каналы связи [9]. В рамках проектирования разработана структурная схема слоистой архитектуры, описывающая логику функционирования игровых комнат, процессы обмена сетевыми пакетами, процедуры динамического обновления текущих игровых состояний, обработку возникающих ошибок и последующего корректного закрытия активных соединений [1][5].

Особое внимание в рамках проекта уделено реализации механизмов разграничения прав доступа к игровым сессиям и сквозной верификации передаваемых данных. Обработка и контроль текущих игровых состояний осуществляются непосредственно на стороне сервера посредством строгой валидации всех входящих сетевых запросов [8]. Изоляция вычислительных процессов на сервере позволяет гарантировать достоверность информации, обрабатываемой в режиме реального времени [6]. Такой комплексный подход исключает компрометацию данных, предотвращает любую возможность подмены пакетов и обеспечивает высокий уровень безопасности сетевого взаимодействия всех участников сессии [10].

Функциональное тестирование программного решения и фиксация закрывающих фреймов при выходе игроков подтвердили корректность работы системы, стабильность сетевых соединений и строгое соответствие структуры передаваемых пакетов исходным техническим требованиям [1]. Разработанный программный комплекс обладает высоким потенциалом практического применения для оптимизации современных веб-сервисов со сложной логикой распределенного взаимодействия [7]. Таким образом, все задачи, поставленные на начальном этапе проектирования, были решены в установленном объёме, что позволило подтвердить работоспособность архитектуры и успешно достичь главной цели работы [2][3].

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] WebSocket API — MDN Web Docs [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
- [2] PostgreSQL 17 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/index.html>
- [3] CSS Grid Layout — MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout
- [4] HTML Standard — MDN Web Docs [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [5] encoding/json — Go encoding/json package (Go Package Documentation) [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/encoding/json>
- [6] database/sql — Go database/sql package (Go Package Documentation) [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/database/sql>
- [7] net/http FileServer — Serving static files (Go Package Documentation) [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/net/http>
- [8] fmt — Go formatted I/O package (Go Package Documentation) [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/fmt>
- [9] Fetch API — MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [10] crypto/sha256 — Go crypto/sha256 package (Go Package Documentation) [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/crypto/sha256>

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

```
=====
FILE: lab_0\cmd\server\main.go
=====

package main

import (
    "log"
    "net/http"

    "poker-duel/internal/auth"
    "poker-duel/internal/database"
    "poker-duel/internal/network"
)

func main() {

    database.InitDB("127.0.0.1", "5432", "postgres", "postgres", "postgres")
    defer database.DB.Close()

    hub := network.NewHub()
    go hub.Run()

    fs := http.FileServer(http.Dir("./public"))

    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        switch r.URL.Path {
        case "/api/auth/register":
            auth.RegisterHandler(w, r)
        case "/api/auth/login":
            auth.LoginHandler(w, r)
        case "/api/user/profile":
            auth.ProfileHandler(w, r)
        case "/api/user/stats":
            auth.StatsHandler(w, r)
        case "/api/game/save-result":
            auth.SaveGameResultHandler(w, r)
        case "/ws":
            network.ServeWS(hub, w, r)
        default:
            fs.ServeHTTP(w, r)
        }
    })

    log.Println("Стеклянный покерный сервер успешно запущен на порту :8080...")
    if err := http.ListenAndServe(":8080", nil); err != nil {
        log.Fatalf("Fatal: %v", err)
    }
}

=====
FILE: lab_0\internal\auth\handler.go
=====

package auth

import (
    "crypto/sha256"
    "encoding/json"
    "fmt"
    "log"

```

```

    "net/http"
    "strconv"
    "strings"

    "poker-duel/internal/database"
)

func hashPassword(password string) string {
    return fmt.Sprintf("%x", sha256.Sum256([]byte(password)))
}

type ErrorResponse struct {
    Error string `json:"error"`
}

type SuccessResponse struct {
    Success bool           `json:"success"`
    Data    map[string]interface{} `json:"data,omitempty"`
}

func sendJSON(w http.ResponseWriter, status int, data interface{}) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(data)
}

func tournamentTrophyDelta(won bool, round int) int {
    if round < 1 {
        round = 1
    }
    if won {
        switch round {
        case 3:
            return 4
        case 2:
            return 2
        default:
            return 1
        }
    }
    switch round {
    case 3:
        return -1
    case 2:
        return -2
    default:
        return -4
    }
}

func RegisterHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        sendJSON(w, http.StatusMethodNotAllowed, ErrorResponse{Error: "Метод не поддерживается"})
        return
    }

    login := strings.TrimSpace(r.FormValue("login"))
    password := r.FormValue("password")

    if login == "" || password == "" {
        sendJSON(w, http.StatusBadRequest, ErrorResponse{Error: "Введите логин и пароль"})
        return
    }
}

```

```

var exists bool
query := `SELECT EXISTS(SELECT 1 FROM users WHERE login = $1)`
database.DB.QueryRow(query, login).Scan(&exists)

if exists {
    sendJSON(w, http.StatusConflict, ErrorResponse{Error: "Логин уже занят"})
    return
}

passwordHash := hashPassword(password)
query = `INSERT INTO users (login, password_hash) VALUES ($1, $2)`
_, err := database.DB.Exec(query, login, passwordHash)
if err != nil {
    sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка базы
данных"})
    return
}

sendJSON(w, http.StatusOK, SuccessResponse{Success: true})
}

func LoginHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        sendJSON(w, http.StatusMethodNotAllowed, ErrorResponse{Error: "Метод не
поддерживается"})
        return
    }

    login := strings.TrimSpace(r.FormValue("login"))
    password := r.FormValue("password")

    if login == "" || password == "" {
        sendJSON(w, http.StatusBadRequest, ErrorResponse{Error: "Введите логин и
пароль"})
        return
    }

    var storedHash string
    var balance, trophies int
    query := `SELECT password_hash, balance, trophies FROM users WHERE login = $1`
    err := database.DB.QueryRow(query, login).Scan(&storedHash, &balance, &trophies)
    if err != nil {
        sendJSON(w, http.StatusUnauthorized, ErrorResponse{Error: "Неверный логин или
пароль"})
        return
    }

    if hashPassword(password) != storedHash {
        sendJSON(w, http.StatusUnauthorized, ErrorResponse{Error: "Неверный логин или
пароль"})
        return
    }

    sendJSON(w, http.StatusOK, SuccessResponse{
        Success: true,
        Data: map[string]interface{}{
            "login":    login,
            "balance":  balance,
            "trophies": trophies,
        },
    })
}

func ProfileHandler(w http.ResponseWriter, r *http.Request) {

```

```

login := r.URL.Query().Get("login")
if login == "" {
    sendJSON(w, http.StatusBadRequest, ErrorResponse{Error: "Логин не указан"})
    return
}

var balance, trophies int
query := `SELECT balance, trophies FROM users WHERE login = $1`
err := database.DB.QueryRow(query, login).Scan(&balance, &trophies)
if err != nil {
    balance = 10000
    trophies = 0
}

sendJSON(w, http.StatusOK, SuccessResponse{
    Success: true,
    Data: map[string]interface{}{
        "login":    login,
        "balance":  balance,
        "trophies": trophies,
    },
})
}

func StatsHandler(w http.ResponseWriter, r *http.Request) {
    login := r.URL.Query().Get("login")
    if login == "" {
        sendJSON(w, http.StatusBadRequest, ErrorResponse{Error: "Логин не указан"})
        return
    }

    var totalGames, wins, maxWin int
    var history []map[string]interface{}

    var userID int
    query := `SELECT id FROM users WHERE login = $1`
    err := database.DB.QueryRow(query, login).Scan(&userID)
    if err == nil {
        query = `SELECT COUNT(*) FROM games_history WHERE winner_id = $1 OR loser_id = $1`
        database.DB.QueryRow(query, userID).Scan(&totalGames)

        query = `SELECT COUNT(*) FROM games_history WHERE winner_id = $1`
        database.DB.QueryRow(query, userID).Scan(&wins)

        query = `SELECT COALESCE(MAX(net_amount), 0) FROM games_history WHERE winner_id = $1`
        database.DB.QueryRow(query, userID).Scan(&maxWin)

        query = `SELECT
            CASE WHEN winner_id = $1 THEN true ELSE false END as won,
            pot,
            net_amount,
            mode,
            round,
            played_at
        FROM games_history
        WHERE winner_id = $1 OR loser_id = $1
        ORDER BY played_at DESC
        LIMIT 10`
        rows, err := database.DB.Query(query, userID)
        if err == nil {
            defer rows.Close()
            for rows.Next() {
                var won bool

```

```

        var pot int
        var netAmount int
        var mode string
        var round int
        var playedAt string
        rows.Scan(&won, &pot, &netAmount, &mode, &round, &playedAt)
        history = append(history, map[string]interface{}{
            "won":      won,
            "pot":      pot,
            "net_amount": netAmount,
            "mode":      mode,
            "round":     round,
        })
    }
}

if totalGames == 0 {
    totalGames = 0
    wins = 0
    maxWin = 0
}

var winPercent int
if totalGames > 0 {
    winPercent = (wins * 100) / totalGames
} else {
    winPercent = 0
}

sendJSON(w, http.StatusOK, SuccessResponse{
    Success: true,
    Data: map[string]interface{}{
        "total_games": totalGames,
        "win_percent": winPercent,
        "max_win":     maxWin,
        "history":     history,
    },
})
}

func SaveGameResultHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        sendJSON(w, http.StatusMethodNotAllowed, ErrorResponse{Error: "Метод не поддерживается"})
        return
    }

    login := strings.TrimSpace(r.FormValue("login"))
    netAmountStr := r.FormValue("net_amount")
    wonStr := r.FormValue("won")
    potStr := r.FormValue("pot")
    mode := r.FormValue("mode")
    roundStr := r.FormValue("round")

    if login == "" {
        sendJSON(w, http.StatusBadRequest, ErrorResponse{Error: "Логин не указан"})
        return
    }

    var netAmount, pot, round int
    netAmount, _ = strconv.Atoi(netAmountStr)
    pot, _ = strconv.Atoi(potStr)
    round, _ = strconv.Atoi(roundStr)
    if round < 1 {

```

```

        round = 1
    }
    won := wonStr == "true"

    tx, err := database.DB.Begin()
    if err != nil {
        log.Println("DB Begin error:", err)
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка базы
данных"})
        return
    }

    var userID int
    query := `SELECT id FROM users WHERE login = $1`
    if err = tx.QueryRow(query, login).Scan(&userID); err != nil {
        log.Println("Get user ID error:", err)
        tx.Rollback()
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Пользователь
не найден"})
        return
    }

    var botID int
    query = `SELECT id FROM users WHERE login = 'bot'`
    if err = tx.QueryRow(query).Scan(&botID); err != nil {
        query = `INSERT INTO users (login, password_hash) VALUES ('bot', 'bot')
RETURNING id`
        if err = tx.QueryRow(query).Scan(&botID); err != nil {
            log.Println("Insert bot error:", err)
            tx.Rollback()
            sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка
создания бота"})
            return
        }
    }

    var winnerID, loserID int
    if won {
        winnerID = userID
        loserID = botID
    } else {
        winnerID = botID
        loserID = userID
    }

    query = `INSERT INTO games_history (winner_id, loser_id, pot, mode, round,
net_amount) VALUES ($1, $2, $3, $4, $5, $6)`
    if _, err = tx.Exec(query, winnerID, loserID, pot, mode, round, netAmount); err !=
nil {
        log.Println("Insert history error:", err)
        tx.Rollback()
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка
сохранения истории"})
        return
    }

    query = `UPDATE users SET balance = balance + $1 WHERE id = $2`
    if _, err = tx.Exec(query, netAmount, userID); err != nil {
        log.Println("Update balance error:", err)
        tx.Rollback()
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка
обновления баланса"})
        return
    }

```

```

    if mode == "Турнир" {
        delta := tournamentTrophyDelta(won, round)
        if delta != 0 {
            query = `UPDATE users SET trophies = GREATEST(trophies + $1, 0) WHERE id =
$2`
            if _, err = tx.Exec(query, delta, userID); err != nil {
                log.Println("Update trophies error:", err)
            }
        }
    }

    var newBalance, newTrophies int
    query = `SELECT balance, trophies FROM users WHERE id = $1`
    if err = tx.QueryRow(query, userID).Scan(&newBalance, &newTrophies); err != nil {
        log.Println("Get new balance error:", err)
        tx.Rollback()
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка
получения данных"})
        return
    }

    if err = tx.Commit(); err != nil {
        log.Println("Commit error:", err)
        sendJSON(w, http.StatusInternalServerError, ErrorResponse{Error: "Ошибка
коммита"})
        return
    }

    sendJSON(w, http.StatusOK, SuccessResponse{
        Success: true,
        Data: map[string]interface{}{
            "balance": newBalance,
            "trophies": newTrophies,
        },
    })
}

=====
FILE: lab_0\internal\database\db.go
=====

package database

import (
    "database/sql"
    "fmt"
    "log"

    _ "github.com/lib/pq"
)

var DB *sql.DB

func InitDB(host, port, user, password, dbname string) {
    connStr := fmt.Sprintf("host=%s port=%s user=%s password=%s dbname=%s
sslmode=disable",
        host, port, user, password, dbname)

    var err error
    DB, err = sql.Open("postgres", connStr)
    if err != nil {
        log.Fatalf("DB Config Error: %v", err)
    }

    if err = DB.Ping(); err != nil {
        log.Fatalf("DB Connection Error: %v. Проверьте запущен ли PostgreSQL.", err)
    }
}

```

```

    createTables()
}

func createTables() {
    queries := []string{
        `CREATE TABLE IF NOT EXISTS users (
            id SERIAL PRIMARY KEY,
            login VARCHAR(50) UNIQUE NOT NULL,
            password_hash VARCHAR(255) NOT NULL,
            balance INT DEFAULT 10000,
            trophies INT DEFAULT 0
        )`,
        `CREATE TABLE IF NOT EXISTS games_history (
            id SERIAL PRIMARY KEY,
            winner_id INT REFERENCES users(id),
            loser_id INT REFERENCES users(id),
            pot INT NOT NULL,
            net_amount INT DEFAULT 0,
            mode VARCHAR(50),
            round INT DEFAULT 0,
            played_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )`,
        `ALTER TABLE games_history ADD COLUMN IF NOT EXISTS round INT DEFAULT 0`,
        `ALTER TABLE games_history ADD COLUMN IF NOT EXISTS net_amount INT DEFAULT 0`,
    }

    for _, q := range queries {
        _, err := DB.Exec(q)
        if err != nil {
            log.Printf("Warning: %v", err)
        }
    }
}

=====
FILE: lab_0\internal\game\bot.go
=====

package game

import "math/rand"

func GetBotAction(difficulty string, botHand []Card, tableCards []Card) string {
    score := 0
    for _, c := range botHand {
        score += c.Score
    }

    switch difficulty {
    case "EASY":
        if rand.Float64() < 0.15 {
            return "fold"
        }
        return "check"

    case "MEDIUM":
        if score < 10 {
            return "fold"
        }
        if score > 22 {
            return "raise"
        }
        return "check"

    case "HARD":
        if score > 24 {

```

```

        return "raise"
    }
    hasHighCardOnTable := false
    for _, tc := range tableCards {
        if tc.Value == "A" || tc.Value == "K" {
            hasHighCardOnTable = true
        }
    }
    if hasHighCardOnTable && rand.Float64() < 0.20 {
        return "raise"
    }
    if score < 14 {
        return "fold"
    }
    return "check"
}

return "check"
}

=====
FILE: lab_0\internal\game\card.go
=====

package game

import (
    "math/rand"
    "time"
)

type Card struct {
    Suit string `json:"suit"`
    Value string `json:"value"`
    Score int `json:"score"`
}

func NewDeck() []Card {
    suits := []string{"♠", "♥", "♦", "♣"}
    values := []string{"2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K",
"A"}

    deck := make([]Card, 0, 52)
    for _, s := range suits {
        for i, v := range values {
            deck = append(deck, Card{Suit: s, Value: v, Score: i + 2})
        }
    }
    rand.Seed(time.Now().UnixNano())
    rand.Shuffle(len(deck), func(i, j int) { deck[i], deck[j] = deck[j], deck[i] })
    return deck
}

=====
FILE: lab_0\internal\game\engine.go
=====

package game

import "sort"

const (
    HighCard = iota
    OnePair
    TwoPair
    ThreeOfKind
    Straight
    Flush
    FullHouse

```

```

    FourOfKind
    StraightFlush
    RoyalFlush
)

func EvaluateHand(hand []Card, table []Card) (int, []int) {
    allCards := append(hand, table...)
    if len(allCards) < 5 {
        return HighCard, getSortedScores(allCards)
    }

    if rank, kickers := checkRoyalFlush(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkStraightFlush(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkFourOfKind(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkFullHouse(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkFlush(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkStraight(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkThreeOfKind(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkTwoPair(allCards); rank != -1 {
        return rank, kickers
    }
    if rank, kickers := checkOnePair(allCards); rank != -1 {
        return rank, kickers
    }

    return HighCard, getSortedScores(allCards)
}

func EvaluateWinner(p1Hand, p2Hand, table []Card) int {
    rank1, kickers1 := EvaluateHand(p1Hand, table)
    rank2, kickers2 := EvaluateHand(p2Hand, table)

    if rank1 > rank2 {
        return 1
    }
    if rank2 > rank1 {
        return 2
    }

    for i := 0; i < len(kickers1) && i < len(kickers2); i++ {
        if kickers1[i] > kickers2[i] {
            return 1
        }
        if kickers2[i] > kickers1[i] {
            return 2
        }
    }
    return 0
}

```

```

func getSortedScores(cards []Card) []int {
    scores := make([]int, len(cards))
    for i, c := range cards {
        scores[i] = c.Score
    }
    sort.Sort(sort.Reverse(sort.IntSlice(scores)))
    return scores
}

func checkRoyalFlush(cards []Card) (int, []int) {
    rank, kickers := checkStraightFlush(cards)
    if rank != -1 && kickers[0] == 14 {
        return RoyalFlush, kickers
    }
    return -1, nil
}

func checkStraightFlush(cards []Card) (int, []int) {
    suitGroups := make(map[string][]Card)
    for _, c := range cards {
        suitGroups[c.Suit] = append(suitGroups[c.Suit], c)
    }

    for _, suitCards := range suitGroups {
        if len(suitCards) >= 5 {
            if rank, kickers := checkStraight(suitCards); rank != -1 {
                return StraightFlush, kickers
            }
        }
    }
    return -1, nil
}

func checkFourOfKind(cards []Card) (int, []int) {
    counts := getScoreCounts(cards)
    for score, count := range counts {
        if count == 4 {
            kickers := []int{score}
            for s := range counts {
                if s != score {
                    kickers = append(kickers, s)
                    break
                }
            }
            return FourOfKind, kickers
        }
    }
    return -1, nil
}

func checkFullHouse(cards []Card) (int, []int) {
    counts := getScoreCounts(cards)
    var three, pair int
    for score, count := range counts {
        if count == 3 {
            three = score
        } else if count >= 2 {
            pair = score
        }
    }
    if three != 0 && pair != 0 {
        return FullHouse, []int{three, pair}
    }
    return -1, nil
}

```

```

}

func checkFlush(cards []Card) (int, []int) {
    suitGroups := make(map[string][]Card)
    for _, c := range cards {
        suitGroups[c.Suit] = append(suitGroups[c.Suit], c)
    }
    for _, suitCards := range suitGroups {
        if len(suitCards) >= 5 {
            return Flush, getSortedScores(suitCards)[:5]
        }
    }
    return -1, nil
}

func checkStraight(cards []Card) (int, []int) {
    scores := make(map[int]bool)
    for _, c := range cards {
        scores[c.Score] = true
    }

    if scores[14] && scores[2] && scores[3] && scores[4] && scores[5] {
        return Straight, []int{5}
    }

    for high := 14; high >= 5; high-- {
        allPresent := true
        for i := 0; i < 5; i++ {
            if !scores[high-i] {
                allPresent = false
                break
            }
        }
        if allPresent {
            return Straight, []int{high}
        }
    }
    return -1, nil
}

func checkThreeOfKind(cards []Card) (int, []int) {
    counts := getScoreCounts(cards)
    for score, count := range counts {
        if count == 3 {
            kickers := []int{score}
            others := []int{}
            for s := range counts {
                if s != score {
                    others = append(others, s)
                }
            }
            sort.Sort(sort.Reverse(sort.IntSlice(others)))
            kickers = append(kickers, others[:2]...)
            return ThreeOfKind, kickers
        }
    }
    return -1, nil
}

func checkTwoPair(cards []Card) (int, []int) {
    counts := getScoreCounts(cards)
    pairs := []int{}
    for score, count := range counts {
        if count == 2 {

```

```

        pairs = append(pairs, score)
    }
}
if len(pairs) >= 2 {
    sort.Sort(sort.Reverse(sort.IntSlice(pairs)))
    kicker := 0
    for s := range counts {
        if s != pairs[0] && s != pairs[1] {
            if s > kicker {
                kicker = s
            }
        }
    }
    return TwoPair, []int{pairs[0], pairs[1], kicker}
}
return -1, nil
}

func checkOnePair(cards []Card) (int, []int) {
    counts := getScoreCounts(cards)
    for score, count := range counts {
        if count == 2 {
            kickers := []int{score}
            others := []int{}
            for s := range counts {
                if s != score {
                    others = append(others, s)
                }
            }
            sort.Sort(sort.Reverse(sort.IntSlice(others)))
            kickers = append(kickers, others[:3]...)
            return OnePair, kickers
        }
    }
    return -1, nil
}

func getScoreCounts(cards []Card) map[int]int {
    counts := make(map[int]int)
    for _, c := range cards {
        counts[c.Score]++
    }
    return counts
}

=====
FILE: lab_0\internal\game\tournament.go
=====

package game

import "fmt"

type Match struct {
    Player1 string
    Player2 string
    Winner  string
}

type TournamentBracket struct {
    ID          string
    Round       int
    QuarterFinal []Match
    SemiFinal   []Match
    Final       Match
    IsFinished  bool
}

```

```

func CreateTournament(players []string) *TournamentBracket {
    for len(players) < 8 {
        players = append(players, fmt.Sprintf("Bot_%d", len(players)+1))
    }

    t := &TournamentBracket{
        ID:    "TOURNEY_GLASS_1",
        Round: 1,
    }

    for i := 0; i < 8; i += 2 {
        t.QuarterFinal = append(t.QuarterFinal, Match{
            Player1: players[i],
            Player2: players[i+1],
        })
    }
    return t
}
=====
FILE: lab_0\internal\network\client.go
=====
package network

import "github.com/gorilla/websocket"

type Client struct {
    Conn *websocket.Conn
    Hub  *Hub
}
=====
FILE: lab_0\internal\network\hub.go
=====
package network

import (
    "encoding/json"
    "fmt"
    "math/rand"
    "net/http"
    "sync"
    "time"

    "poker-duel/internal/game"

    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{CheckOrigin: func(r *http.Request) bool { return true }}

type Hub struct {
    Clients    map[*websocket.Conn]string
    Rooms      map[string]*GameRoom
    ArenaQueue []*websocket.Conn
    SpinQueue  []*websocket.Conn
    mu         sync.Mutex
}

func NewHub() *Hub {
    return &Hub{
        Clients: make(map[*websocket.Conn]string),
        Rooms:   make(map[string]*GameRoom),
    }
}

```

```

func (h *Hub) Run() {
    for {
        time.Sleep(2 * time.Second)
        h.mu.Lock()
        if len(h.ArenaQueue) >= 2 {
            p1, p2 := h.ArenaQueue[0], h.ArenaQueue[1]
            h.ArenaQueue = h.ArenaQueue[2:]
            h.startMatch("ARENA", p1, p2, 100, 200)
        }
        if len(h.SpinQueue) >= 2 {
            p1, p2 := h.SpinQueue[0], h.SpinQueue[1]
            h.SpinQueue = h.SpinQueue[2:]
            h.startMatch("SPIN", p1, p2, 100, 200)
        }
        h.mu.Unlock()
    }
}

func (h *Hub) startMatch(mode string, conn1, conn2 *websocket.Conn, smallBlind,
bigBlind int) {
    var room *GameRoom
    if mode == "FRIEND" {
        room, _ = h.getPlayerRoom(conn1)
        if room == nil {
            return
        }
        room.Hub = h
        for _, p := range room.Players {
            p.Cards = []game.Card{}
            p.Chips = 10000
            p.Bet = 0
            p.Folded = false
            p.IsTurn = false
            p.AllIn = false
        }
    } else {
        roomID := fmt.Sprintf("R_%d", rand.Intn(100000))
        room = &GameRoom{
            ID:          roomID,
            SmallBlind: smallBlind,
            BigBlind:      bigBlind,
            Hub:           h,
        }
        room.Players = []*PlayerState{
            {
                Conn:    conn1,
                Name:    h.Clients[conn1],
                Cards: []game.Card{},
                Chips: 10000,
                Bet:    0,
                Folded: false,
                IsTurn: false,
                AllIn:  false,
            },
            {
                Conn:    conn2,
                Name:    h.Clients[conn2],
                Cards: []game.Card{},
                Chips: 10000,
                Bet:    0,
                Folded: false,
                IsTurn: false,
                AllIn:  false,
            },
        },
    }
}

```

```

    }
    h.Rooms[roomID] = room
}
room.Deck = []game.Card{}
room.Table = []game.Card{}
room.Pot = 0
room.CurrentTurn = 0
room.GamePhase = "waiting"
room.DealerPos = 0
if room.SmallBlind == 0 {
    room.SmallBlind = smallBlind
    room.BigBlind = bigBlind
}

multiplier := 2
if mode == "SPIN" {
    multipliers := []int{2, 4, 6, 10}
    multiplier = multipliers[rand.Intn(4)]
}

msg1, _ := json.Marshal(map[string]interface{}{
    "type": "game_start", "room": room.ID, "mode": mode, "multiplier": multiplier,
})
msg2, _ := json.Marshal(map[string]interface{}{
    "type": "game_start", "room": room.ID, "mode": mode, "multiplier": multiplier,
})

room.Players[0].Conn.WriteMessage(websocket.TextMessage, msg1)
room.Players[1].Conn.WriteMessage(websocket.TextMessage, msg2)

room.StartGame()
}

func (h *Hub) getPlayerRoom(conn *websocket.Conn) (*GameRoom, int) {
    for _, room := range h.Rooms {
        for i, p := range room.Players {
            if p.Conn == conn {
                return room, i
            }
        }
    }
    return nil, -1
}

func (h *Hub) processTimeout(roomID string) {
    h.mu.Lock()
    defer h.mu.Unlock()

    room, ok := h.Rooms[roomID]
    if !ok {
        return
    }

    if room.GamePhase == "finished" || room.GamePhase == "showdown" {
        return
    }

    room.HandleTimeout()
}

func ServeWS(hub *Hub, w http.ResponseWriter, r *http.Request) {

```

```

conn, err := upgrader.Upgrade(w, r, nil)
if err != nil {
    return
}

username := r.URL.Query().Get("user")
hub.mu.Lock()
hub.Clients[conn] = username
hub.mu.Unlock()

go func() {
    defer func() {
        hub.mu.Lock()
        delete(hub.Clients, conn)
        conn.Close()
        hub.mu.Unlock()
    }()
    for {
        _, message, err := conn.ReadMessage()
        if err != nil {
            break
        }
        var req map[string]interface{}
        json.Unmarshal(message, &req)

        hub.mu.Lock()

        if req["action"] == "search_arena" {
            hub.ArenaQueue = append(hub.ArenaQueue, conn)
        } else if req["action"] == "search_spin" {
            hub.SpinQueue = append(hub.SpinQueue, conn)
        } else if req["action"] == "create_friend" {
            code := req["code"].(string)
            room := &GameRoom{ID: code, GamePhase: "waiting", Hub: hub}
            room.Players = append(room.Players, &PlayerState{
                Conn: conn,
                Name: hub.Clients[conn],
                Cards: []game.Card{},
                Chips: 10000,
                Bet: 0,
                Folded: false,
                IsTurn: false,
                AllIn: false,
            })
            hub.Rooms[code] = room
        } else if req["action"] == "join_friend" {
            code := req["code"].(string)
            if room, ok := hub.Rooms[code]; ok && len(room.Players) < 2 {
                if len(room.Players) == 1 {
                    room.Players = append(room.Players, &PlayerState{
                        Conn: conn,
                        Name: hub.Clients[conn],
                        Cards: []game.Card{},
                        Chips: 10000,
                        Bet: 0,
                        Folded: false,
                        IsTurn: false,
                        AllIn: false,
                    })
                    hub.startMatch("FRIEND", room.Players[0].Conn,
room.Players[1].Conn, 50, 100)
                }
            }
        } else if req["action"] == "fold" {
            room, playerId := hub.getPlayerRoom(conn)

```

```

        if room != nil && playerIdx != -1 && room.Players[playerIdx].IsTurn {
            room.PlayerFold(playerIdx)
        }
    } else if req["action"] == "check" {
        room, playerIdx := hub.getPlayerRoom(conn)
        if room != nil && playerIdx != -1 && room.Players[playerIdx].IsTurn {
            if room.Players[playerIdx].Bet == room.CurrentBet {
                room.PlayerCheck(playerIdx)
            } else {
                room.PlayerCall(playerIdx)
            }
        }
    } else if req["action"] == "call" {
        room, playerIdx := hub.getPlayerRoom(conn)
        if room != nil && playerIdx != -1 && room.Players[playerIdx].IsTurn {
            room.PlayerCall(playerIdx)
        }
    } else if req["action"] == "bet" {
        room, playerIdx := hub.getPlayerRoom(conn)
        if room != nil && playerIdx != -1 && room.Players[playerIdx].IsTurn {
            amount := 0
            if amt, ok := req["amount"].(float64); ok {
                amount = int(amt)
            }
            room.PlayerBet(playerIdx, amount)
        }
    } else if req["action"] == "raise" {
        room, playerIdx := hub.getPlayerRoom(conn)
        if room != nil && playerIdx != -1 && room.Players[playerIdx].IsTurn {
            amount := 0
            if amt, ok := req["amount"].(float64); ok {
                amount = int(amt)
            }
            room.PlayerRaise(playerIdx, amount)
        }
    }
}

hub.mu.Unlock()
}
}()
}

=====
FILE: lab_0\internal\network\room.go
=====

package network

import (
    "encoding/json"
    "poker-duel/internal/game"
    "time"

    "github.com/gorilla/websocket"
)

type PlayerState struct {
    Conn    *websocket.Conn
    Name    string
    Cards   []game.Card
    Chips   int
    Bet     int
    Folded  bool
    IsTurn  bool
    AllIn   bool
}

```

```

type GameRoom struct {
    ID            string
    Players       []*PlayerState
    Deck          []game.Card
    Table         []game.Card
    Pot           int
    CurrentTurn   int
    GamePhase     string
    SmallBlind    int
    BigBlind      int
    CurrentBet    int
    LastAction    string
    DealerPos     int
    Timer         *time.Timer
    TimerSeconds  int
    Hub           *Hub
    PlayersActed int
}

func (room *GameRoom) Broadcast(message interface{}) {
    for _, p := range room.Players {
        if p.Conn != nil {
            room.SendToPlayer(p, message)
        }
    }
}

func (room *GameRoom) SendToPlayer(player *PlayerState, baseMessage interface{}) {
    playerInfos := make([]map[string]interface{}, len(room.Players))
    for i, p := range room.Players {
        cards := []map[string]interface{}{}
        if room.GamePhase == "showdown" || room.GamePhase == "finished" || p.Conn ==
player.Conn {
            for _, c := range p.Cards {
                cards = append(cards, map[string]interface{}{"suit": c.Suit, "value":
c.Value})
            }
            playerInfos[i] = map[string]interface{}{
                "name":    p.Name,
                "chips":    p.Chips,
                "bet":      p.Bet,
                "folded":   p.Folded,
                "is_turn":  p.IsTurn,
                "all_in":   p.AllIn,
                "cards":   cards,
            }
        }

        tableCards := []map[string]interface{}{}
        for _, c := range room.Table {
            tableCards = append(tableCards, map[string]interface{}{"suit": c.Suit, "value":
c.Value})
        }

        fullMsg := make(map[string]interface{})
        if baseMap, ok := baseMessage.(map[string]interface{}); ok {
            for k, v := range baseMap {
                fullMsg[k] = v
            }
        }
        fullMsg["type"] = "game_state"
        fullMsg["room"] = room.ID
        fullMsg["phase"] = room.GamePhase
    }
}

```

```

    fullMsg["pot"] = room.Pot
    fullMsg["table_cards"] = tableCards
    fullMsg["players"] = playerInfos
    fullMsg["current_turn"] = room.CurrentTurn
    fullMsg["current_bet"] = room.CurrentBet
    fullMsg["last_bet"] = room.CurrentBet
    fullMsg["last_action"] = room.LastAction
    fullMsg["dealer_pos"] = room.DealerPos

    msg, _ := json.Marshal(fullMsg)
    player.Conn.WriteMessage(websocket.TextMessage, msg)
}

func (room *GameRoom) GetGameState() map[string]interface{} {
    return map[string]interface{}{}
}

func (room *GameRoom) markZeroChipPlayersAllIn() {
    for _, p := range room.Players {
        if !p.Folded && p.Chips <= 0 {
            p.AllIn = true
        }
    }
}

func (room *GameRoom) resetBettingRound() {
    for _, p := range room.Players {
        p.Bet = 0
        p.IsTurn = false
    }
    room.CurrentBet = 0
    room.LastAction = ""
    room.PlayersActed = 0
    for _, p := range room.Players {
        if !p.Folded && p.AllIn {
            room.PlayersActed++
        }
    }
}

func (room *GameRoom) shouldResolveRound() bool {
    activeCount := 0
    allMatched := true
    allInCount := 0

    for _, p := range room.Players {
        if p.Folded {
            continue
        }
        activeCount++
        if !p.AllIn && p.Bet != room.CurrentBet {
            allMatched = false
        }
        if p.AllIn {
            allInCount++
        }
    }

    if activeCount <= 1 {
        return false
    }

    if !allMatched || room.PlayersActed < 2 {
        return false
    }
}

```

```

    if allInCount >= 1 {
        room.dealRemainingCards()
        room.GamePhase = "showdown"
        room.DetermineWinner()
        return true
    }

    room.NextPhase()
    return true
}

func (room *GameRoom) StartGame() {
    room.Deck = game.NewDeck()
    room.Table = []game.Card{}
    room.Pot = 0
    room.GamePhase = "preflop"
    room.CurrentBet = room.BigBlind
    room.LastAction = ""
    room.TimerSeconds = 20

    for i, p := range room.Players {

        p.Cards = append([]game.Card{}, room.Deck[i*2:(i+1)*2]...)
        p.Bet = 0
        p.Folded = false
        p.AllIn = false
        p.IsTurn = false
    }

    room.PlayersActed = 0

    sbPos := room.DealerPos
    bbPos := (room.DealerPos + 1) % 2

    room.Players[sbPos].Bet = room.SmallBlind
    room.Players[sbPos].Chips -= room.SmallBlind
    room.Players[bbPos].Bet = room.BigBlind
    room.Players[bbPos].Chips -= room.BigBlind
    room.Pot = room.SmallBlind + room.BigBlind

    room.markZeroChipPlayersAllIn()

    if room.Players[sbPos].AllIn {
        room.CurrentTurn = bbPos
    } else {
        room.CurrentTurn = sbPos
    }
    room.Players[room.CurrentTurn].IsTurn = true
    room.PlayersActed = 0
    for _, p := range room.Players {
        if !p.Folded && p.AllIn {
            room.PlayersActed++
        }
    }

    if room.shouldResolveRound() {
        return
    }

    room.StartTimer()
    room.Broadcast(room.GetGameState())
}

```

```

func (room *GameRoom) NextPhase() {
    room.StopTimer()

    switch room.GamePhase {
    case "preflop":
        room.GamePhase = "flop"
        room.Table = append(room.Table, room.Deck[4:7]...)
    case "flop":
        room.GamePhase = "turn"
        room.Table = append(room.Table, room.Deck[7])
    case "turn":
        room.GamePhase = "river"
        room.Table = append(room.Table, room.Deck[8])
    case "river":
        room.GamePhase = "showdown"
        room.DetermineWinner()
        return
    }

    room.markZeroChipPlayersAllIn()

    room.resetBettingRound()

    sbPos := room.DealerPos
    room.CurrentTurn = sbPos
    room.Players[room.CurrentTurn].IsTurn = true

    if room.shouldResolveRound() {
        return
    }

    room.StartTimer()
    room.Broadcast(room.GetGameState())
}

func (room *GameRoom) dealRemainingCards() {
    if room.GamePhase == "preflop" && len(room.Table) == 0 {
        room.Table = append(room.Table, room.Deck[4:7]...)
    }
    if len(room.Table) < 4 {
        room.Table = append(room.Table, room.Deck[7])
    }
    if len(room.Table) < 5 {
        room.Table = append(room.Table, room.Deck[8])
    }
}

func (room *GameRoom) clearPlayerTurns() {
    for _, p := range room.Players {
        p.IsTurn = false
    }
    room.CurrentTurn = -1
}

func (room *GameRoom) PlayerFold(playerIndex int) {
    room.StopTimer()
    room.Players[playerIndex].Folded = true
    room.Players[playerIndex].IsTurn = false
    room.LastAction = "fold"

    activePlayers := 0
    winnerIndex := 0

```

```

for i, p := range room.Players {
    if !p.Folded {
        activePlayers++
        winnerIndex = i
    }
}

if activePlayers == 1 {
    room.clearPlayerTurns()
    room.Players[winnerIndex].Chips += room.Pot
    room.GamePhase = "finished"
    room.SwitchDealer()
    room.Broadcast(room.GetGameState())

    gameOver := false
    var loserIndex int
    for i, p := range room.Players {
        if p.Chips <= 0 {
            gameOver = true
            loserIndex = i
            break
        }
    }

    if gameOver {
        for i, p := range room.Players {
            if p.Conn != nil {
                won := i != loserIndex
                msg, _ := json.Marshal(map[string]interface{}{
                    "type": "game_over",
                    "won": won,
                })
                p.Conn.WriteMessage(websocket.TextMessage, msg)
            }
        }
        return
    }

    time.AfterFunc(2*time.Second, func() {
        room.StartGame()
    })
    return
}

room.NextTurn()
}

func (room *GameRoom) PlayerCheck(playerIndex int) {
    room.StopTimer()
    room.Players[playerIndex].IsTurn = false
    room.PlayersActed++
    room.LastAction = "check"
    room.NextTurn()
}

func (room *GameRoom) PlayerCall(playerIndex int) {
    room.StopTimer()
    callAmount := room.CurrentBet - room.Players[playerIndex].Bet
    if callAmount >= room.Players[playerIndex].Chips {
        callAmount = room.Players[playerIndex].Chips
        room.Players[playerIndex].AllIn = true
    }
}

```

```

    room.Players[playerIndex].Chips -= callAmount
    room.Players[playerIndex].Bet += callAmount
    room.Pot += callAmount
    if room.Players[playerIndex].Chips == 0 {
        room.Players[playerIndex].AllIn = true
    }
    room.Players[playerIndex].IsTurn = false
    room.PlayersActed++
    room.LastAction = "call"

    room.NextTurn()
}

func (room *GameRoom) PlayerBet(playerIndex int, amount int) {
    room.StopTimer()
    totalBet := amount
    betAmount := totalBet - room.Players[playerIndex].Bet

    if betAmount >= room.Players[playerIndex].Chips {
        betAmount = room.Players[playerIndex].Chips
        room.Players[playerIndex].AllIn = true
        totalBet = room.Players[playerIndex].Bet + betAmount
    }

    room.Players[playerIndex].Chips -= betAmount
    room.Players[playerIndex].Bet = totalBet
    room.Pot += betAmount
    if room.Players[playerIndex].Chips == 0 {
        room.Players[playerIndex].AllIn = true
    }
    room.CurrentBet = totalBet
    room.Players[playerIndex].IsTurn = false
    room.PlayersActed = 1
    room.LastAction = "bet"

    room.NextTurn()
}

func (room *GameRoom) PlayerRaise(playerIndex int, amount int) {
    room.StopTimer()

    totalBet := amount
    raiseAmount := totalBet - room.Players[playerIndex].Bet

    if raiseAmount >= room.Players[playerIndex].Chips {
        raiseAmount = room.Players[playerIndex].Chips
        room.Players[playerIndex].AllIn = true
        totalBet = room.Players[playerIndex].Bet + raiseAmount
    }

    if raiseAmount < 0 {
        raiseAmount = 0
        totalBet = room.Players[playerIndex].Bet
    }

    room.Players[playerIndex].Chips -= raiseAmount
    room.Players[playerIndex].Bet = totalBet
    room.Pot += raiseAmount
    if room.Players[playerIndex].Chips == 0 {
        room.Players[playerIndex].AllIn = true
    }
    room.CurrentBet = totalBet
    room.Players[playerIndex].IsTurn = false
    room.PlayersActed = 1

```

```

        room.LastAction = "raise"

        room.NextTurn()
    }

func (room *GameRoom) NextTurn() {
    room.markZeroChipPlayersAllIn()

    if room.shouldResolveRound() {
        return
    }

    room.CurrentTurn = (room.CurrentTurn + 1) % 2
    for room.Players[room.CurrentTurn].Folded || room.Players[room.CurrentTurn].AllIn {
        room.CurrentTurn = (room.CurrentTurn + 1) % 2

        if room.Players[room.CurrentTurn].Folded ||
room.Players[room.CurrentTurn].AllIn {

            anyActive := false
            for _, p := range room.Players {
                if !p.Folded && !p.AllIn {
                    anyActive = true
                    break
                }
            }
            if !anyActive {

                room.dealRemainingCards()
                room.GamePhase = "showdown"
                room.DetermineWinner()
                return
            }
        }
    }
    room.Players[room.CurrentTurn].IsTurn = true

    room.StartTimer()
    room.Broadcast(room.GetGameState())
}

func (room *GameRoom) DetermineWinner() {
    room.StopTimer()
    room.clearPlayerTurns()

    var winners []int
    bestRank := -1
    var bestKickers []int

    for i, p := range room.Players {
        if p.Folded {
            continue
        }

        rank, kickers := game.EvaluateHand(p.Cards, room.Table)
        if rank > bestRank {
            bestRank = rank
            bestKickers = kickers
            winners = []int{i}
        } else if rank == bestRank {
            better := false
            equal := true
            for j := 0; j < len(kickers) && j < len(bestKickers); j++ {

```

```

        if kickers[j] > bestKickers[j] {
            better = true
            equal = false
            break
        } else if kickers[j] < bestKickers[j] {
            equal = false
            break
        }
    }
    if better {
        winners = []int{i}
        bestKickers = kickers
    } else if equal {
        winners = append(winners, i)
    }
}

if len(room.Players) == 2 && !room.Players[0].Folded && !room.Players[1].Folded {
    p0Bet := room.Players[0].Bet
    p1Bet := room.Players[1].Bet
    minBet := p0Bet
    if p1Bet < minBet {
        minBet = p1Bet
    }
    mainPot := minBet * 2
    sidePot := room.Pot - mainPot

    if len(winners) == 1 {
        winner := winners[0]
        loser := 1 - winner
        if room.Players[winner].Bet == minBet && room.Players[loser].Bet > minBet {
            room.Players[winner].Chips += mainPot
            room.Players[loser].Chips += sidePot
        } else {
            room.Players[winner].Chips += room.Pot
        }
    } else {
        room.Players[0].Chips += mainPot / 2
        room.Players[1].Chips += mainPot / 2
        if sidePot > 0 {
            if p0Bet > p1Bet {
                room.Players[0].Chips += sidePot
            } else if p1Bet > p0Bet {
                room.Players[1].Chips += sidePot
            } else {
                room.Players[0].Chips += sidePot / 2
                room.Players[1].Chips += sidePot / 2
            }
        }
    }
} else {
    prizePerWinner := room.Pot / len(winners)
    for _, idx := range winners {
        room.Players[idx].Chips += prizePerWinner
    }
}

room.SwitchDealer()
room.GamePhase = "finished"
room.Broadcast(room.GetGameState())

gameOver := false
var loserIndex int

```

```

    for i, p := range room.Players {
        if p.Chips <= 0 {
            gameOver = true
            loserIndex = i
            break
        }
    }
}

if gameOver {

    for i, p := range room.Players {
        if p.Conn != nil {
            won := i != loserIndex
            msg, _ := json.Marshal(map[string]interface{}{
                "type": "game_over",
                "won": won,
            })
            p.Conn.WriteMessage(websocket.TextMessage, msg)
        }
    }
    return
}

time.AfterFunc(2*time.Second, func() {
    room.StartGame()
})
}

func (room *GameRoom) SwitchDealer() {
    room.DealerPos = (room.DealerPos + 1) % 2
}

func (room *GameRoom) StartTimer() {
    room.Timer = time.AfterFunc(time.Duration(room.TimerSeconds)*time.Second, func() {

        if room.Hub != nil {
            room.Hub.processTimeout(room.ID)
        }
    })
}

func (room *GameRoom) StopTimer() {
    if room.Timer != nil {
        room.Timer.Stop()
        room.Timer = nil
    }
}

func (room *GameRoom) HandleTimeout() {
    player := room.Players[room.CurrentTurn]
    if player.Folded || player.AllIn {
        return
    }

    if player.Bet == room.CurrentBet {
        room.PlayerCheck(room.CurrentTurn)
    } else {
        room.PlayerFold(room.CurrentTurn)
    }
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы